

Opus: Photonic Rail-Optimized Fabric in ML Datacenters

Eric Ding
Cornell University
Ithaca, NY, USA

Bhaskar Kataria
Cornell University
Ithaca, NY, USA

Barry Lyu
University of Michigan
Ann Arbor, MI, USA

Rachee Singh
Cornell University
Ithaca, NY, USA

Abstract

Rail-optimized network fabrics have become the de facto datacenter scale-out fabric for large-scale ML training. However, the use of high-radix electrical switches to provide all-to-all connectivity in rails imposes substantial power and cost. We propose a rethinking of the rail abstraction by retaining its communication semantics, but realizing it using optical circuit switches. The key challenge is that optical switches support one-to-one connectivity at a time, limiting the fan-out of traffic in ML workloads using hybrid parallelisms. We overcome this through *parallelism-driven rail reconfiguration*, which exploits the non-overlapping communication phases of different parallelism dimensions. This time-multiplexes a single set of physical ports across circuit configurations tailored to each phase within a training iteration. We design and implement Opus, a control plane that orchestrates this in-job reconfiguration of photonic rails at parallelism phase boundaries, and evaluate it on a physical OCS testbed, the Perlmutter supercomputer, and in simulation at up to 2,048 GPUs. Our results show that photonic rails can achieve over 23 $\times$ network power reduction and 4 $\times$ cost savings while incurring only modest training overhead at production-relevant OCS reconfiguration latencies.

Keywords

Reconfigurable Networks, Optical Circuit Switching, Large Language Model, Collective Communication

1 Introduction

The design of datacenter interconnect fabrics has a significant impact on the scalability and efficiency of large-scale machine learning (ML) systems. A wide range of general datacenter fabric designs have been proposed in the past decade, including electrical [1, 24, 26, 75, 76] and photonic networks [11, 15, 23, 25, 82]. More recently, the rapid growth of ML training workloads has driven a shift toward ML-centric datacenter fabric designs [29, 30, 32, 81, 84, 86, 93]. Among the many proposals, one datacenter topology has seen broad adoption: the rail-optimized fabric [22, 60]. The rail fabric explicitly aligns with the communication patterns of hybrid parallelisms in distributed ML by wiring together “rails” of GPUs—sets of GPUs with identical ranks across multiple high-bandwidth (or *scale-up*) domains—to the same switch (Figure 1). This design can achieve congestion-free communication for common collective operations like ALLREDUCE and ALLGATHER in distributed ML pipelines [60].

But this performance comes at a steep cost. Each rail switch connects GPUs of the same rank in all scale-up domains, resulting in networks built from high-radix packet switches. These switches

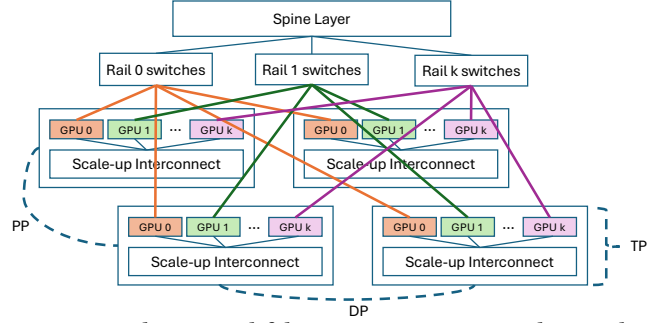


Figure 1: Rail-optimized fabrics. We propose to replace packet switches (shown as Rail 0, Rail 1 etc.) with optical circuit switches. We make the case for retaining the illusion of full connectivity between GPU ranks connected to the same optical rail switch using in-job reconfiguration.

are not only costly, but also power-hungry. Switch ASIC processing and transceiver electrical-optical conversions contribute to the energy and complexity burden of the fabric [10, 54, 65]. As ML clusters scale to tens of thousands of GPUs, the networking fabric’s power footprint grows proportionally, consuming an increasingly significant share of the datacenter’s total energy budget. Reducing the power consumed by the network is therefore not just a cost optimization, it is becoming an operational necessity for sustainably scaling ML infrastructure.

This paper asks whether it is possible to retain the desirable properties of rail-optimized fabrics while improving their energy and cost efficiency. Rather than redesigning the datacenter topology [7, 32, 36, 77, 81, 84], we pursue a different direction to answer the question: we propose to replace electrical packet switches in the rail with reconfigurable optical circuit switches (OCSes) which consume orders of magnitude less power than their electrical counterparts [46, 86]. We call the resulting design a *photonic rail-optimized fabric*. Our proposal draws inspiration from recent successes in optical ML fabrics [29, 93] but departs from them by preserving the widely adopted rail design, while fundamentally changing how data is switched within it.

However, the shift from packet switching to circuit switching in rails is not straightforward to achieve in practice. Electrical rail switches enable *all-to-all* packet-level connectivity among GPUs in the same rail, whereas OCSes only offer one-to-one circuit connectivity, *i.e.*, a matching between GPUs, at a given time. This breaks a key invariant of rail-optimized designs—all-to-all connectivity among the same-rank GPUs across all scale-up domains (Figure 1). Without this property, common collective operations used in hybrid

parallelisms can no longer be efficiently implemented. The core technical challenge, then, is: *how to retain the abstraction of rail communication despite the limitations of photonic switching?*

Opus’s key observation is that most communication in distributed ML is predictable and structured [21, 84]. Collectives are issued in known sequences, organized by parallelism type (e.g., tensor, data, pipeline) [55]. We exploit this structure to break the illusion of requiring full connectivity by *reconfiguring the photonic fabric between collectives within the job*. To our knowledge, this is the first proposal to reconfigure fabrics between ML collectives, while other proposals reconfigure once prior to the job start [29, 84, 93], or reconfigure during workloads specifically for one type of parallelism [36].

To realize the vision of photonic rails, we develop Opus, a control plane that orchestrates parallelism-driven rail reconfiguration. Opus introduces a control layer between ML training frameworks and collective communication libraries. Collective communication libraries act as clients to Opus and issue provisional intents to communicate, while Opus interfaces with network orchestrators to reconfigure the optical fabric at parallelism phase boundaries. Opus’s design ensures that reconfiguration is both safe—circuits are never torn down while traffic is in flight—and efficient—speculative provisioning hides reconfiguration latency within natural idle windows between parallelism phases.

Opus enables a principled quantification of the tradeoff at the heart of photonic rails: *how much ML training performance must be sacrificed to achieve the power and cost savings of photonic rails?* Through experiments on a physical OCS hardware testbed, emulation on the Perlmutter supercomputer (up to 64 GPUs), and large-scale simulations (up to 2,048 GPUs), we show that this tradeoff is remarkably favorable. At production-relevant OCS reconfiguration latencies (≤ 100 ms), Opus incurs less than 6.7% increase in iteration time while achieving up to 4.27 $\times$ reduction in networking infrastructure cost and over 23 $\times$ reduction in network power consumption compared to electrical rail-optimized fabrics. This shows that the performance penalty of in-job reconfiguration is a modest price for an order-of-magnitude improvement in the energy efficiency of ML datacenter networks.

2 Our Proposal: Electrical Rails $\rightarrow$ Optical Rails

Training ML models at scale requires a combination of parallelism strategies across thousands of GPUs [12, 14, 70, 73]. To make this feasible, ML systems leverage multiple, co-existing parallelisms (Table 4 in Appendix §A). These include data parallelism (DP, and fully sharded data parallelism or FSDP), pipeline parallelism (PP), tensor parallelism (TP, often with sequence parallelism or SP), context parallelism (CP), and expert parallelism (EP) [14, 16, 22, 31, 39, 70, 73, 91]. Each parallelism incurs communication that differs in: (1) **data volume**: ranging from full model size in DP to per-layer activations in TP; (2) **start time**: some collectives occur in the forward pass, others only in backpropagation; (3) **frequency**: some fire once per layer, others once per micro-batch; and (4) **communication pattern**: from symmetrical collectives, where every rank participates, to peer-to-peer asymmetrical SEND/RECV (Appendix §A). Importantly, the communication operations from different parallelisms are not ordered arbitrarily: they follow strict

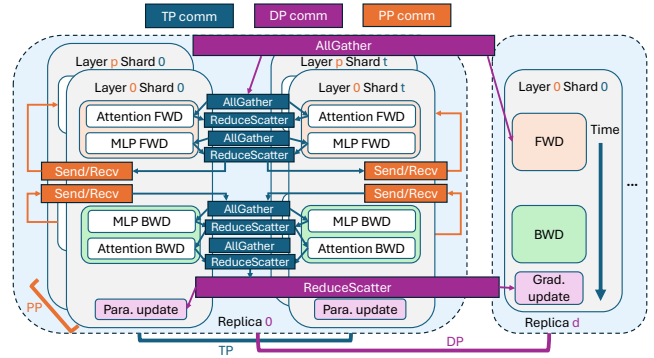


Figure 2: Traffic in a training iteration with 3D parallelism.

dependencies defined by the model’s compute graph. Figure 2 shows these dependencies in a 3D-parallel training step.

Rail-optimized topologies organize the scale-out network into multiple independent *rails* [60]. Each rail connects GPUs of the same local rank across all scale-up domains (e.g., an NVIDIA DGX [57] or HGX [59] node), providing a dedicated, congestion-free path for collective communication on the scale-out network, like DP and PP communication [60].

Figure 1 illustrates how a 3D parallelism strategy (i.e., DP, TP and PP) maps onto a rail fabric. Frequent and latency-sensitive TP collectives are confined within the high-bandwidth scale-up domains, whereas PP and DP collectives traverse the slower scale-out network since they occur less frequently and can be overlapped with compute [22, 66]. The rail abstraction allows these scale-out collectives to occur without oversubscribing links in the datacenter fabric since every rail provides a dedicated, congestion-free path among GPUs of the same rank across domains.

Limitations of packet-switched rails. Despite their performance benefits, today’s rail fabrics suffer from scalability challenges. Optical fibers connect scale-up domains to the rail switches. These links terminate at transceivers that interface with server NICs and packet switch ports. Each packet switch introduces optical-electrical-optical (OEO) conversions, adding energy and latency overhead to the data path. These conversions, coupled with the switch ASIC’s work—packet queueing, header parsing, and TCAM lookups—consume significant energy [61, 90]. Moreover, while link speeds (e.g., 400 Gbps) continue to scale, ASIC processing speed has not kept pace. As a result, packet-switched fabrics now represent a bottleneck in both power efficiency and bandwidth scalability, especially in the face of LLM training jobs that require hundreds of rails and thousands of GPUs [9].

Replacing rail packet switches with OCSes. We re-architect the rail-based datacenter fabric to replace electrical packet switches with reconfigurable optical circuit switches (OCSes). These optical switches can form end-to-end circuits without OEO conversions and eliminate switch ASICs entirely from the datapath. The resulting architecture improves on the traditional rail fabric by reducing energy consumption of the network, reducing datapath latency, and scaling bandwidth without incurring ASIC bottlenecks [40]. Importantly, our proposal retains the physical structure of the existing rail-optimized topology: the scale-up domains, cabling, and GPU-to-rail mapping, all remain unchanged. There is no multi-tier

electrical rail or spine. Instead, each rail becomes a flat, photonic point-to-point fabric. Cross-rank communication can still be supported via forwarding through the high-bandwidth interconnect in scale-up (e.g., PXN [49, 83]). The control plane remains electrical and host-driven.

State-of-the-art OCS technology. Mature switch technologies like MEMS-based OCSes already have characteristics required by our proposed design, including millisecond reconfiguration and switch radix in the hundreds [4, 34, 36, 40, 64]. Note that our proposal differs from recent silicon photonic architectures proposed by NVIDIA and Broadcom [8, 53], which still rely on electrical switching ASICs but use co-packaged optics (CPO) instead of pluggable optical transceivers.

3 Optical Rails: Challenges and Opportunities

Replacing electrical rail switches with OCSes is non-trivial. In ML jobs with hybrid parallelisms, each GPU participates in multiple *communication groups*, i.e., logical constructs managed by collective communication libraries like NCCL [48], where each group is associated with a different parallelism axis. This results in a high communication degree per GPU. For example, in a 3D-parallel job using ring-based collective communication (e.g., ALLREDUCE), each GPU requires two neighbors per ring across three parallelism dimensions, yielding a minimum degree of six. Electrical rails support high degrees of communication naturally when a GPU’s dedicated NIC connects to an electrical packet switch, enabling all-to-all connectivity between connected GPUs, at all times.

3.1 Challenges

In contrast, an OCS can establish only as many simultaneous circuits as a GPU has physical NICs, which is far fewer than the communication degree that hybrid parallelisms demand. Thus, photonic rails face three key challenges: **C1:** Low degree restricts collectives to using only ring algorithms that are bandwidth-efficient but incur high latency [72]. Latency-optimized strategies like tree-based or recursive-doubling collectives cannot be used [71, 72, 80]; **C2:** The number of parallelisms is constrained as some parallelisms (e.g., CP) need dedicated connectivity [35]; **C3:** Distributing a GPU’s NIC ports across communication groups allocates only a fraction of NIC bandwidth to each collective, underutilizing the provisioned bandwidth.

A concrete example. Consider training a model with 3D parallelism (TP, DP, and PP) on DGX H200 nodes (scale-up) connected by an optical rail (scale-out). Each GPU has dedicated access to a ConnectX-7 NIC, which can operate in three configurations: 1×400 Gbps port, 2×200 Gbps ports, or 4×100 Gbps ports [50, 56]. TP is confined to the scale-up domain, while DP and PP collectives traverse the scale-out optical rail. For the DP collectives, each GPU needs to connect to two neighbors in a ring for ring-based ALLREDUCE. For PP collectives, each GPU needs to connect to two neighbors for sending and receiving activations. Connections for both of these parallelisms will be made between the GPU’s NIC ports and the rail OCS. If we use the 4-port NIC configuration, we can assign two ports each to DP and PP to have sufficient degree for ring-based collectives to relevant neighbors. This halves the bandwidth available to collectives of each parallelism (C3). Moreover, the low per-group degree forces the use of ring-based algorithms (C1), and

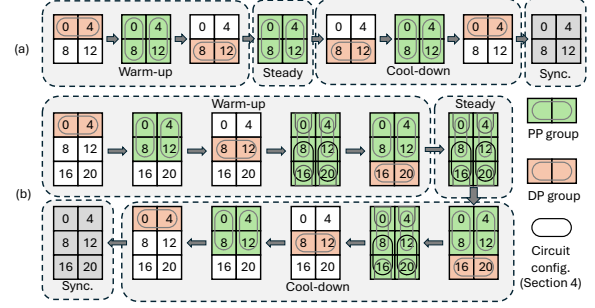


Figure 3: Communication pattern for PP and FSDP in one iteration, split based on the warm-up, steady, and cool-down stages of the pipeline (4 rails in total, only rail 0 is shown, TP is hidden). (a) PP=2, FSDP=2. (b) PP=3, FSDP=2.

adding a fourth scale-out parallelism such as CP is infeasible without additional NICs (C2). The root cause of these limitations is that static circuit-based connectivity of the OCS forces all parallelisms to share a fixed set of physical ports simultaneously.

A note on prior work. One workaround to these limitations is to multiplex parallelisms over shared physical links across scale-up domains. But this introduces a new set of problems: forwarding traffic via intermediate GPUs to reach the destination inflates latency and incurs a bandwidth tax [42]. Prior OCS-based ML fabrics have sidestepped these constraints by adding many more NICs per GPU [84, 86, 87], restricting the OCS fabric to a single parallelism while relying on electrical switches for the rest [36], or constructing a fixed torus before the job starts [29], which still suffers from C1 and C3 [32, 81].

3.2 Opportunities

We observe that, although a GPU belongs to many communication groups, it does not use them all at once. Collectives from different parallelisms follow strict data dependencies defined by the model’s computational graph (Figure 2): DP collectives occur after backpropagation completes, PP Send/Recv operations interleave with forward and backward passes, and so on. These dependencies create brief *windows* of communication inactivity between parallelism phases.

If the OCS can be reconfigured within such a window, a single set of physical ports can be time-multiplexed across all parallelism dimensions—each phase receiving the *full* NIC bandwidth and a circuit topology tailored to its collective, eliminating C1–C3 entirely. This motivates a departure from prior reconfigurable network designs, which target microsecond- or nanosecond-scale adaptation for general datacenter traffic [2, 4, 15, 23, 34, 42, 74, 82] and are ill suited to the repetitive, high-volume collective patterns of ML workloads [84]. We instead propose to reconfigure at the granularity of ML collectives themselves—a coarser but far more natural unit of adaptation for distributed training.

Understanding the communication pattern. Figure 3(a) shows the communication pattern in a 3D-parallel workload. Stage 0 first performs micro-batch 0 forward pass (overlapped with per-layer ALLGATHER to collect the next layer’s parameters), and sends the activation to stage 1 hosted by rank 8 through a SEND/RECV call along the pipeline dimension. Once the SEND/RECV call is finished, rank 8 computes the forward pass, while doing ALLGATHER. Then,

it performs backpropagation for micro-batch 0, followed by pipeline SEND/RECV. REDUCE/SCATTER calls are issued after partial gradients are updated. During the optimizer step, several short ALLREDUCE calls are issued for synchronization. We observe that the DP traffic does not overlap with PP traffic. Figure 3(b) shows the pattern for PP=3. The data dependency between operations, and PyTorch’s lazy DTensor operation (e.g., the first ALLGATHER call for stage 1 only starts when it receives the activation from stage 0), dictate the sequential order between PP and DP traffic, though collectives from two dimensions are issued in different CUDA streams. We can then define the window as the idle time between two consecutive parallelism phases $P1$ and $P2$, which are two distinctive sets of communication groups:

$$T_{\text{window}} = \min_{comm_j \in P2} T_{comm_j_start} - \max_{comm_i \in P1} T_{comm_i_end},$$

where $comm_j \neq comm_i$ for all $comm_i \in P1$. In addition,

$$T_{comm_j_start} = \max_{rank_x \in comm_j} T_{rank_x_comm_j_start},$$

where $rank_x$ participates in $comm_j$, since the collective starts only when the slowest rank joins. $T_{comm_i_end}$ is the end time of the $comm_i$, the same for all participating ranks.

Empirical findings. To study the window sizes, we run an LLM training workload using TorchTitan [35] on the Perlmutter super-computer [45]. We conduct four experiments: **(Exp. 1)** Llama-3-8B with TP=4 (intra-node), FSDP=2, PP=2, global batch size=16, sequence length=8192. **(Exp. 2)** DeepSeek-V3-236B with TP=4, FSDP=8, PP=8, EP=32, global batch size=64, sequence length=1024. **(Exp. 3)** Llama-3-8B with TP=4, FSDP=8, PP=2, global batch size=64, sequence length=8192. **(Exp. 4)** Llama-3-70B with TP=4, FSDP=4, PP=8, global batch size=32, sequence length=1024.

We find that reconfiguration windows are typically millisecond-scale. Figure 4(a,b) plots the CDF of window durations for the first workload and groups windows by the traffic volume of the following communication operation $comm_j$. More than 75% of windows are longer than 1 ms, and the distributions are similar across rails. The highest-volume operation, REDUCE/SCATTER, is preceded by the longest windows, averaging roughly 1,000 ms. For the workloads in Figure 4(c), the median window duration changes because of varying collective sizes and different parallelism configurations. Even so, the median duration remains above 1 ms.

To understand the window duration in larger deployments, we simulate a frontier-scale workload under strong scaling, with a large global batch during frontier model pre-training (GPT-style [41], PP=4, TP=32, global batch size 15,360, sequence length 4096 [14], 96 layers, hidden dimension 8192) on B200 architecture using ASTRA-sim [69] (Figure 4(d)). Compared to previous experiments, the larger local batches increase computation load, lengthening the windows. The window size decreases under strong scaling: the median window duration falls from 187 ms at 2,048 GPUs to 23.9 ms at 16,384 GPUs. We further estimate and plot the GPU utilization (fraction of time spent on computation), assuming network reconfiguration is performed at the beginning of each window in Figure 5:

$$Util (\%) = \frac{T_{\text{compute}}}{T_{\text{compute}} + T_{\text{non_overlap_comm}} + T_{\text{stall}}} \times 100, \quad (1)$$

$$T_{\text{stall}} = \sum_i \max(0, T_{\text{reconfig}} - T_{\text{window},i}).$$

Here, $T_{\text{non_overlap_comm}}$ is the non-overlapped communication time, T_{reconfig} is the reconfiguration latency, and $T_{\text{window},i}$ is the duration for window i . Utilization is flat while the reconfiguration latency stays below most of the windows and degrades due to slower switches or larger deployments. With fast OCSES ($T_{\text{reconfig}} < 10$ ms) [46, 86], the GPU utilization remains high for frontier model training. Additional results for smaller batch sizes (e.g., 3,072 used in the initial pre-training phase [14]) are included in Appendix §B. For general workloads, the number of windows in one iteration can be determined by Equation 2, assuming FSDP is used and the TP domain does not exceed scale-up. In the equation, n_{layer} is the number of model layers, $n_{\text{microbatch}}$ is the number of microbatches per global batch, and PP is the pipeline parallel degree.

Summary. The non-overlapping communication pattern across parallelism dimensions creates an opportunity where each time window between parallelism phases can be used to configure circuits tailored to the upcoming collective operation. The size of individual windows depends on multiple factors, including the model size, batch size, sequence length, number of layers per pipeline stage, parallelism dimensions and communication group size. Finally, our findings indicate that circuit configuration can have limited impact on ML application performance if the reconfiguration delay is on the order of milliseconds, allowing it to be hidden within these natural idle periods between parallelism phases.

$$\begin{aligned} \text{Window count} \leq & \underbrace{4 \cdot (PP - 1)}_{\text{PP and FSDP fwd/bwd interleave}} + \underbrace{2 \cdot \frac{n_{\text{layer}}}{PP} - 1}_{\text{CP/EP and FSDP 1st microbatch fwd interleave}} + \underbrace{4 \cdot n_{\text{microbatch}}}_{\text{CP/EP and PP fwd/bwd interleave}} \\ & + \underbrace{2 \cdot n_{\text{microbatch}} \cdot \left(2 \cdot \frac{n_{\text{layer}}}{PP} - 1\right)}_{\text{CP and EP fwd/bwd interleave}} + \underbrace{4}_{\text{PP warm-up, steady, cool-down, sync, and state transition}} \end{aligned} \quad (2)$$

4 Parallelism-driven Rail Reconfiguration

Photonic rails limit the connection degree of GPUs, in turn limiting effective hybrid parallelism (§3). Therefore, to realize photonic rails in ML datacenters where jobs employ hybrid parallelisms, it is essential to reconfigure the scale-out optical fabric *within* each job to ensure each parallelism phase has sufficient degree of connectivity.

In-job topology reconfiguration must satisfy three objectives: (1) minimize reconfiguration delay, (2) reduce reconfiguration frequency to maximize circuit uptime, and (3) avoid conflicts between new circuits and ongoing traffic in the network. Meeting these objectives requires control logic that has knowledge of the parallelism structure of the ML workload. Therefore, this control logic must reside in the application layer, where parallelism-level hints are directly accessible. Moreover, by recognizing changes in parallelism phases (e.g., DP $\rightarrow$ TP), the control logic can *provision* circuits speculatively—initiating reconfiguration as soon as the previous kernel completes (Figure 6)—and suppress redundant reconfigurations when consecutive collectives belong to the same parallelism dimension (Figure 3). Note that our design choice to operate at the application layer departs from prior reconfigurable network controllers that operate on packets or flows at layers 2 and 3 [2, 4, 15, 23, 34, 42, 74, 82].

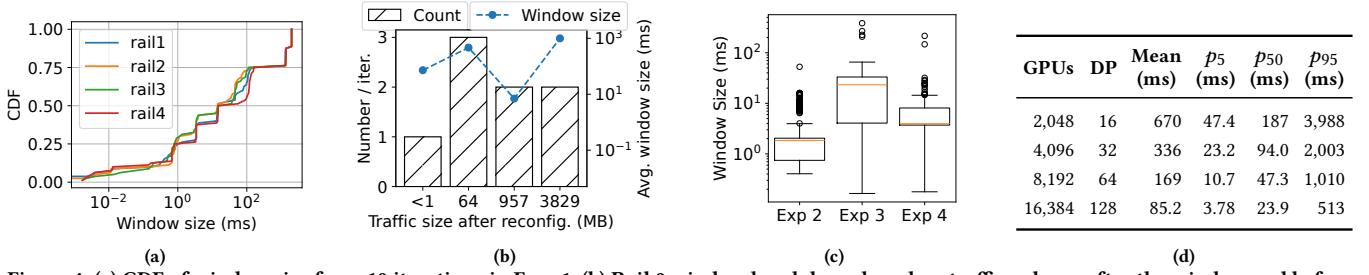


Figure 4: (a) CDF of window size from 10 iterations in Exp. 1. (b) Rail 0 window breakdown based on traffic volume after the window and before the next window, in one iteration of Exp. 1. <1MB: ALLREDUCE synchronization calls, 64MB: PP SEND/RECV, 957MB: DP ALLGATHER, 3829MB: DP REDUCESCATTER. (c) Rail 0 window-size box plot for the step latency for Exp. 2–4. (d) Window size statistics for a GPT-style workload [41] under DP strong scaling at frontier scale running on B200.

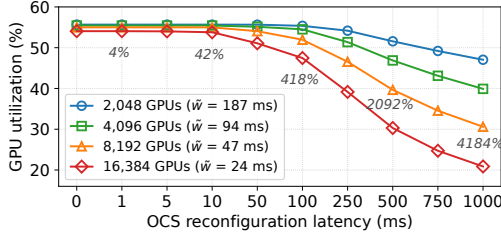


Figure 5: GPU utilization vs. OCS reconfiguration latency for a frontier workload on B200 (Figure 4(d)). $\bar{w}$ is a workload’s median window. Percentages show the latency as a fraction of the median window at the largest (16,384-GPU) scale.

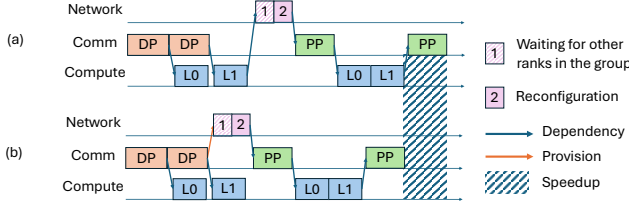


Figure 6: Reconfiguration during the warm-up stage of ranks 0 and 4, (a) without provisioning, (b) with provisioning.

4.1 Opus System Architecture

We present Opus, an application-layer control plane for parallelism-driven reconfiguration of optical circuit switches in rail-based datacenter fabrics. A distributed ML training job alternates between *parallelism phases*: contiguous intervals during which all scale-out communication belongs to a single parallelism dimension (e.g., all collectives are DP ALLGATHER or REDUCESCATTER, or all operations are PP SEND/RECV). Phase boundaries, i.e., transitions like PP $\rightarrow$ DP, are the natural points at which the rail’s circuit topology can be safely reconfigured without disrupting in-flight traffic.

Opus exploits this structure by treating rail connectivity as an allocatable resource whose shape is derived from the upcoming collective’s communication pattern, and reconfigures the optical fabric accordingly. To achieve this, Opus introduces a control layer between ML training frameworks and collective communication libraries, comprising three components (Figure 7): the **Opus shim**,

which runs on every GPU rank within a job and intercepts collective calls, detects phase boundaries, and issues reconfiguration requests; the **Opus controller**, spawned once per job, which synchronizes reconfiguration demands across ranks and forwards topology requests to the network; and the **Opus network orchestrator**, spawned once per rail, which translates topology requests into OCS port-to-port programming commands.

In Opus, the data path for inter-GPU communication is simple: GPU $\rightarrow$ NIC $\rightarrow$ optical fabric $\rightarrow$ NIC $\rightarrow$ GPU. Opus’s *control path* decides which physical circuits are active at any instant. This separation keeps the datapath free of packet switching while allowing control logic to exploit parallelism-level hints available in the ML stack. Together, the shim and controller present an *illusion of all-to-all* GPU connectivity.

The Opus Shim. One shim instance runs per GPU rank, so the number of shim instances equals the total number of GPUs in the job. Each shim maintains: (i) a *phase table*, populated during profiling, that records the sequence of parallelism phases and the communication operations that demarcate them; (ii) an operation index *idx* that tracks progress through the communication schedule; and (iii) the current parallelism phase. Each shim handles all $N_{parallel}$ communication groups to which its GPU belongs.

The shim intercepts every collective communication call from the distributed ML framework and extracts per-call metadata: the *communication group* (participant ranks and their ordering), the *operation type* (collectives like ALLGATHER and ALLREDUCE, or point-to-point operations like SEND/RECV), and the *traffic size*. Using this metadata, the shim classifies the operation as either a management operation (e.g., a barrier at the end of a forward-backward step) or a data-carrying operation, and decides: (1) **Network selection**—RDMA GPU backend (scale-out) or CPU frontend network (Algorithm 1, lines 3, 16); (2) **Reconfiguration trigger**—whether the operation marks a new parallelism phase requiring a topology change (Algorithm 1, line 7); and (3) **Timing**—reconfigure *before* (on-demand, Algorithm 1, line 8) or *after* the preceding operation (provisioning, Algorithm 2, line 3).

When reconfiguration is needed, the shim issues a blocking `topo_write` call to the controller, carrying three fields:

(*comm_group_id*, *idx*, *asym_comm_way*),

with *asym_comm_way* indicating the way (or stage) in an asymmetrical communication group (e.g., PP).

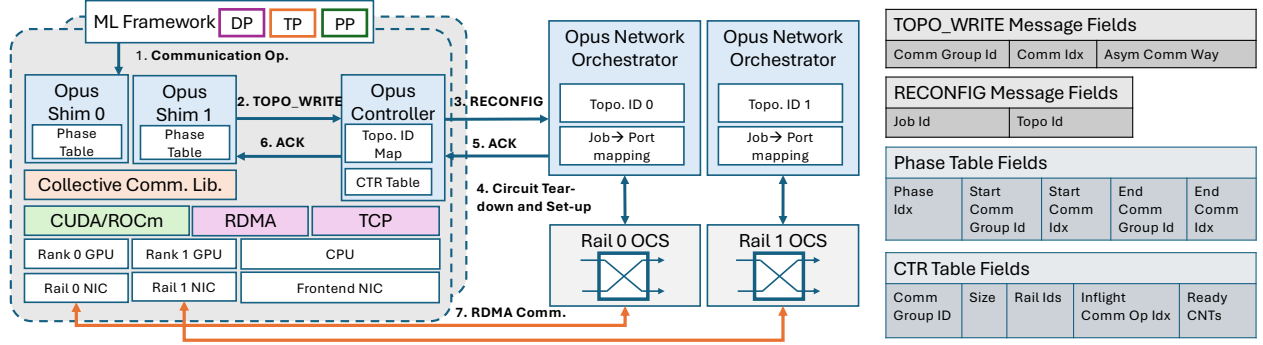


Figure 7: Opus system architecture.

Algorithm 1 Pre-communication Control Logic

```

1: procedure PRE_COMM(comm_op)
2:   if comm_op is scale_up or management then
3:     select scale-up or CPU front-end network
4:     return
5:   end if
6:   wait till topology is free
7:   shift ← PHASE_CHANGE_BEFORE
8:   if mode = DEFAULT and (shift or comm_op is asymm.) then
9:     TOPO_WRITE(comm_group.id, idx, asym_way)
10:  end if
11:  if shift then
12:    comm_stage ← comm_stage + 1
13:    set topology busy
14:  end if
15:  idx ← idx + 1
16:  select GPU backend
17:  register POST_COMM
18: end procedure

```

The Opus Controller. One controller instance is spawned per job. It maintains a *CTR Table* (Figure 7) with metadata for every communication group: sockets to each shim, the group size, the rail IDs used by the group, the index of the in-flight communication operation, and a *ready counter* that tracks how many ranks have issued a *topo_write* for the current operation. For a job with three parallelism dimensions of sizes P_1, P_2, P_3 , there are $P_1P_2 + P_2P_3 + P_3P_1$ communication groups in total. The controller also maintains a per-rail topology identifier, *topo_id_i*, which encodes the current connectivity requirement of the job on rail *i*.

Topology ID encoding. The *topo_id* is a decimal integer whose digit positions correspond to the *ways* (stages) of the job’s asymmetrical parallelism (e.g., the P_{asym} stages of PP). Each digit value encodes which parallelism dimension currently “owns” the connectivity for that stage: 0 denotes PP (asymmetrical), and digits 1, 2, ... denote symmetrical parallelisms (DP, CP, EP, etc.). Up to 9 symmetrical parallelisms can be encoded per digit.

Example (Figure 8). Consider a job with PP=3, DP=2, CP=2. The PP dimension is asymmetrical, so the *topo_id* has three digits—one per pipeline stage. When all three stages perform DP ALLREDUCE,

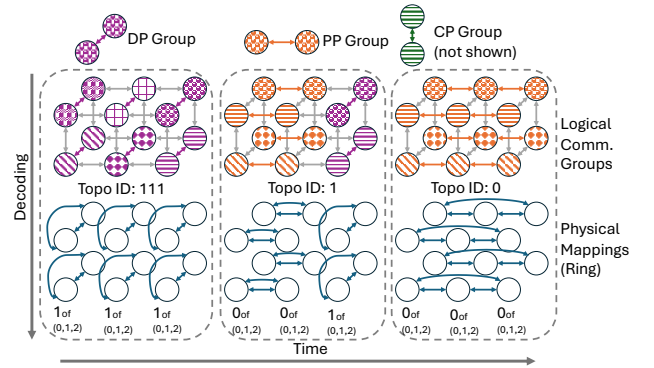


Figure 8: Translating parallelism shifts to topology reconfigurations for a workload with PP=3, DP=2, CP=2. PP is the asymmetrical parallelism, and DP & CP are symmetrical parallelisms. Opus topology orchestrator updates sub-mappings of affected ways in PP when the topology needs to be reconfigured. The mapping constructs per-group rings.

the topology is *topo_id* = 111 (all digits set to 1, representing DP). When stage 0 and 1 transition to PP SEND/RECV while stage 2 remains on DP, the digits for stage 0 and 1 toggle to 0, yielding *topo_id* = 001. The orchestrator detects the changed digits and reprograms only the sub-mapping for stage 0 and 1.

Runtime synchronization. The controller serves as a synchronization barrier. When a rank joins with a *topo_write*, the controller increments the ready counter for the corresponding communication group and operation index. Once the counter reaches the group size, the controller: (1) updates the *topo_id* based on *asym_comm_way* and *comm_group_id*; (2) forwards the updated *topo_id* to one or more network orchestrators; (3) waits for ACKs confirming reconfiguration is complete; (4) ACKs all ranks in the group; and (5) clears the ready counter for operation *idx*.

The Opus Network Orchestrator. One orchestrator instance manages each rail’s OCSes. For every job *j* on rail *i*, the orchestrator stores the current *topo_id* and the port assignments for that job.

Sub-mapping decomposition. A naïve approach stores all possible port-to-port mappings for every job, requiring $O(\sum_j N_{parallel,j}^{P_{asym,j}} \cdot N_{rank,j})$ space, where $N_{parallel,j}$ is the number of parallelism dimensions, $P_{asym,j}$ is the asymmetrical parallelism degree (e.g., PP stage count), and $N_{rank,j}$ is the number of ranks of job *j* on this rail.

Opus reduces this by decomposing the mapping into $P_{\text{asym},j}$ sub-mappings—one per pipeline stage—each covering $N_{\text{rank},j}/P_{\text{asym},j}$ ports. The total storage becomes $O(\sum_j N_{\text{parallel},j} \cdot N_{\text{rank},j})$, and each reconfiguration event programs only $O(N_{\text{rank},j}/P_{\text{asym},j})$ ports rather than all $N_{\text{rank},j}$.

Reconfiguration dispatch. Upon receiving a topo_id from the controller, the orchestrator compares each digit against the current topo_id for that job. A changed digit triggers reprogramming of the corresponding sub-mapping (Figure 8).

Two cases can arise: (i) *symmetrical-to-asymmetrical shift* (ALLGATHER in CP followed by SEND/RECV in PP): at most two digits change, and the two corresponding sub-mappings are rewired along the PP dimension. (ii) *asymmetrical-to-symmetrical shift* (e.g., SEND/RECV in PP followed by ALLREDUCE in DP at stage m): the orchestrator shifts sub-mapping m and the sub-mappings of the stages that were previously connected to m . Sub-mappings belonging to other jobs, or sub-mappings of the same job that are not involved in the current transition, remain undisturbed. A non-blocking OCS, like the one we use in our hardware experiments, programs only the affected ports without impacting other circuits.

4.2 Reconfiguration Protocol

Opus provides two safety guarantees (G) and two performance optimizations (O):

- **G1.** A communication operation is never carried on a topology that is under reconfiguration.
- **G2.** A reconfiguration is never initiated while a communication operation is in flight on the affected sub-mappings.
- **O1.** Number of reconfigurations per iteration is minimized.
- **O2.** Reconfiguration delay is hidden whenever possible.

Event ordering. G1 and G2 prevent packet loss. They are enforced through a lock in the network backend. When the shim detects a phase boundary after a communication operation, it acquires the lock, issues a topo_write , and releases the lock only after receiving ACK from the controller. No communication operation that uses GPU backend network can proceed on the locked shim until the lock is released.

Profiling parallelism phases. Optimization O1 requires knowing the parallelism phase structure in advance. During the first several training iterations (e.g., 5 steps), the shim profiles the complete traffic pattern by recording the sequence of collectives, their parallelism associations, and inter-collective timing. From this trace, the shim identifies phase boundaries—transitions between communication groups belonging to different parallelism dimensions—and populates the phase table. After profiling, the shim issues reconfiguration requests only at phase boundaries, suppressing redundant reconfigurations when consecutive operations belong to the same phase. The phase table also enables locking (the shim knows whether the next operation requires a different topology) and provisioning, which we describe next.

Provisioning. Optimization O2 exploits the predictability of the communication schedule. In provisioning mode, the shim issues speculative reconfiguration requests immediately after the *last* operation of the current phase completes (Algorithm 2, line 5), rather than waiting until the *first* operation of the next phase arrives

Algorithm 2 Post-communication Control Logic

```

1: procedure POST_COMM( $\text{comm\_group}$ ,  $\text{comm\_op}$ )
2:    $\text{shift} \leftarrow \text{PHASE\_CHANGE\_AFTER}$ 
3:   if  $\text{mode} = \text{PROVISIONING}$  and ( $\text{shift}$  or  $\text{comm\_op}$  is
      asymm.) then
4:     ( $\text{n\_group\_id}$ ,  $\text{n\_idx}$ )  $\leftarrow$  GET_NEXT_COMM
5:     TOPO_WRITE( $\text{n\_group\_id}$ ,  $\text{n\_idx}$ ,  $\text{asym\_way}$ )
6:   end if
7:   if  $\text{shift}$  then
8:     set topology free
9:   end if
10: end procedure

```

(Figure 6). This allows the orchestrator to program the OCS during the idle window between phases. The total delay exposed to communication across an iteration is $\sum_i \max(0, T_{\text{reconfig}} - T_{\text{window},i})$. Windows with $T_{\text{window},i} \geq T_{\text{reconfig}}$ fully hide reconfiguration and contribute no delay. As shown in §3, most phase-transition windows exceed 1 ms, accommodating OCS reconfiguration times of fast switching technologies [46, 86].

Handling asymmetrical parallelism. For symmetrical parallelisms (e.g., DP, CP, EP), all ranks in a communication group participate in every collective, so locking and provisioning operate at phase granularity: the shim locks before the first operation of a new phase and unlocks after the last. PP is *asymmetrical*: different pipeline stages may execute different parallelism phases concurrently. For example, stage 0 may be processing a forward pass with DP REDUCESCATTER while stage 1 is idle, waiting for activations.

This heterogeneity means that a single per-phase lock is insufficient. Instead, for PP communication, the Opus shim performs locking and provisioning at *per-operation* granularity: every PP SEND/RECV triggers a topo_write from both the sending and receiving stage, regardless of the previous communication pattern. This ensures the reconfiguration intent is always conveyed to the OCS fabric, even when one stage has been idle. A reconfiguration triggered by PP SEND/RECV changes at most two digits in the topo_id , corresponding to the two communicating stages.

Handling ALLToALL. To handle the large fan-out in expert-parallel ALLToALL, Opus sets different topologies (ring orderings) across different rails. The topologies are chosen to maximize the connectivity and reduce forwarding cost in the scale-out network. Concretely, Opus uses divergent per-rail topologies so that their union forms a maximally connected scale-out topology for each EP group, letting most pairs exchange in a single scale-out hop through forwarding in the high-bandwidth scale-up domain. Details about the topology design and the routing protocol are in Appendix §D.

Handling communication faults. Opus handles transient failures through timeout-based detection. If a reconfiguration ACK is not received within a configurable timeout, the controller retries. Persistent failures trigger fallback to a *giant ring*—a static circuit connecting all ranks—that provides basic connectivity at reduced bandwidth. The training framework is notified by the shim, and can checkpoint and restart affected ranks. For OCS hardware failures, Opus leverages rail redundancy: each GPU connects to multiple

rails through the scale-up interconnect, so traffic can be rerouted through alternate rails at the cost of increased contention.

We provide complete pseudocode in Appendix §C.

4.3 Implementation

We implement Opus as a custom PyTorch distributed backend. A training job enables Opus by passing `backend="opus"` to PyTorch’s `init_process_group`, or equivalently, by setting a single configuration flag in frameworks built on PyTorch’s distributed API (e.g., `enable_opus_backend` in TorchTitan [35]). No changes to model code or parallelism constructs (DP/PP/TP/CP/EP) are required to use Opus. Opus intercepts collective calls through PyTorch’s `ProcessGroup` abstraction and routes them through the Opus shim, which delegates data transport to NCCL.

Shim runtime. The shim uses two threads to separate control-plane work from data-plane execution. The main thread runs `pre_comm` logic—network selection, phase-shift detection, and reconfiguration requests—before dispatching each collective to NCCL. A dedicated callback thread runs `post_comm` as soon as the CUDA stream signals completion of a collective, enabling provisioning to overlap with the idle window between phases. In the common case (no phase shift), both threads execute without blocking the data plane. When a phase transition occurs, the threads acquire a lock that serializes reconfiguration with communication (§4).

Traffic profiling. During the first five training steps, the shim profiles the communication pattern by recording each collective’s communication group and the operation index within a parallelism phase. The resulting traces are stored in the *phase table*, a cache that the shim consults in subsequent steps to detect phase boundaries, suppress redundant reconfigurations, and drive provisioning decisions.

Network orchestrator interface. The orchestrator communicates with OCSes through a vendor-neutral switch-driver interface supporting protocols such as TL1, SCPI, and NETCONF. Configuration commands specify source–destination port mappings. Hardware dependence is isolated behind this interface, so the same controller logic can target a production OCS or a software emulator during evaluation.

Code. The shim runtime and controller comprise approximately 4,000 lines of C++, built against PyTorch 2.10.0, NCCL 2.28, and CUDA 12.8. The network orchestrator for our hardware testbed is 300 lines of Python, using a TL1 wrapper to interface with the switch. We test Opus with TorchTitan [35] and provide simulation tooling, built on top of ASTRA-sim [69] for reconfigurable rail topologies. Both are open-sourced at <https://github.com/opusfabric/Opus>.

5 Evaluating Opus

We evaluate Opus across three complementary scales: a small-scale hardware testbed validates Opus’s reconfiguration mechanisms on real OCS hardware; medium-scale emulation on the Perlmutter supercomputer (up to 64 GPUs) measures end-to-end training overhead under logically emulated circuit switches; and large-scale simulation in Astra-Sim (up to 2,048 GPUs) with a custom reconfigurable network backend evaluates Opus’s scalability and sensitivity to OCS reconfiguration latency and link bandwidth.

5.1 Lab Hardware Evaluation

We evaluate Opus on our hardware testbed (Figure 9(a)) that comprises one Polatis Series 6000 optical circuit switch with 64 optical LC/UPC ports [63] and four NVIDIA L40 GPU servers, each housing 2 L40 GPUs and 2 ConnectX-6 Dx Ethernet NICs. Each NIC exposes two 100G ports fitted with Cisco QSFP-100G-LR4 single-mode transceivers (1310 nm), yielding 16 transceivers and 32 fiber cables connecting the servers to the switch in a two-rail topology. All components are commercially available and use standard networking interfaces and protocols. We use RoCEv2 for RDMA transport and NCCL 2.28 for collective communication. We made no modifications to the RoCEv2 protocol, Mellanox driver, or NCCL library. We set the RDMA `RETRY_CNT` to 7 so that queue pairs do not enter an error state during reconfiguration events, allowing NCCL to reuse existing connections after circuits are reconfigured.

Workload execution. We train a Llama-3 model (6 layers) with DP=2, TP=2, PP=2 on the hardware testbed with Opus. Figure 9(b) shows the communication operations across two pipeline stages within one forward-backward step. TP collectives are handled by the intra-server PCIe interconnect and do not traverse the optical fabric. Note that on a production-grade testbed, the intra-server interconnect (e.g., NVLink instead of PCIe) is much faster than the optical fabric. On the scale-out network, Opus triggers 4 reconfiguration events per step, shifting the rail topology between DP and PP configurations. At each reconfiguration, Opus allocates the full 200 Gbps per-GPU bandwidth (2×100G ports) to the active parallelism dimension, confirming that photonic rails can provide full-bandwidth, per-phase connectivity.

Reconfiguration timeline. We measure reconfiguration latency at each layer of the network stack to identify bottlenecks (Figure 9(c) and 9(d)). Optical Rx power returns to baseline within 200 ms of the switching command, though `ethtool`’s PHY-layer sampling interval (100 ms) prevents resolving the exact OCS switching time [37]. Once the firmware reports link-up, the Linux kernel `netdev` carrier transitions within $\approx 70\text{--}80\ \mu\text{s}$, and RDMA applications resume transmitting within an additional $\approx 100\text{--}200\ \text{ms}$.

Key Takeaway from hardware experiments.

The dominant delay in reconfiguring on our hardware testbed is in the NIC and transceiver firmware: after Rx power stabilizes, the proprietary firmware takes ≈ 6 seconds to report link-up via `mlx5_query_vport_state` [52, 58]. Disabling auto-negotiation (PHY_AUTO_NEG via `mlxconfig` [47] and `autoneg` via `ethtool` [37]) reduces this to ≈ 3 seconds. This firmware bottleneck is not fundamental to photonic rails or Opus—it reflects the firmware’s assumption that link-state changes are rare failure events rather than routine reconfigurations. With firmware support for fast link-up [42], the end-to-end reconfiguration latency would be bounded by the OCS switching time ($\approx 25\ \text{ms}$ for our Polatis switch [63], and $< 10\ \text{ms}$ for state-of-the-art OCSes [46]).

5.2 Emulation of Opus on a Supercomputer

To evaluate Opus at larger scale without the NIC firmware bottleneck found in §5.1, we emulate optical circuit switching on the Perlmutter supercomputer [45]. The Opus shim and controller run unmodified. We replace the network orchestrators with logical circuit switches that (1) enforce one-to-one connectivity, (2) inject

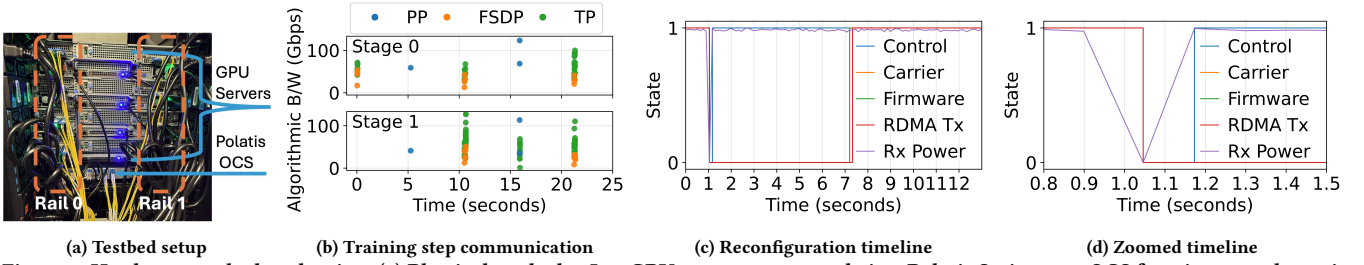


Figure 9: Hardware testbed evaluation. (a) Physical testbed: 4 L40 GPU servers connected via a Polatis Series 6000 OCS forming two photonic rails. (b) Communication within one training step (Llama-3, DP=2, TP=2, PP=2) across two pipeline stages, showing Opus reconfiguring between DP and PP phases. Algorithmic bandwidth is the collective buffer size divided by execution time. (c,d) Timeline of system signals during optical switch reconfiguration: Control is the control-plane command duration; Carrier is the Linux netdev carrier state; Firmware is the link status from `mlx5_query_vport_state`; RDMA Tx indicates successful (1) or failed (0) sends; Rx Power shows normalized received optical power. The zoomed view (d) shows that the OCS and optical link recover within ≈ 200 ms (with measurement overhead), but NIC firmware delays link-up by ≈ 6 s.

Config	Model	Global Batch Size	Seq. Length	Parallelism (TP, FSDP, PP)
1	Llama-3-8B	16	8192	(4, 2, 2)
2	Llama-3-8B	64	8192	(4, 8, 2)
3	DeepSeek-V3-16B	8	2048	(4, 1, 4)

Table 1: Workload configurations. The number of microbatches equals the PP degree. Config. 3 uses only PP in the scale-out and requires no in-job reconfiguration.

configurable reconfiguration delays, and (3) support non-blocking reconfiguration of a subset of circuits without disrupting the rest. Each scale-up domain has 4 NVIDIA A100 GPUs (4 rails) with 3rd-gen NVLink (800 Gbps per direction) and HPE Cray Cassini NICs (200 Gbps).

Our testing framework allows toggling Opus on and off. With Opus disabled, workloads run on the native PyTorch distributed backend with NCCL and GPUDirect RDMA over Perlmutter’s Slingshot-11 fabric, which is a fully connected, electrical-packet-switched (EPS) network. This provides a direct comparison between Opus on emulated photonic rails and the native software stack on ideal EPS. We evaluate three modes: (1) *Native*: TorchTitan on EPS without Opus, (2) *Opus*: emulated reconfigurable topology with configurable OCS delays, and (3) *Opus + Provisioning*: speculative reconfiguration of topology is enabled. Table 1 lists the workload configurations. All metrics are averaged over 5 training steps after 5 warm-up steps. The four topology orchestrators and the controller run on node 0. The emulation results do not reflect the congestion-free benefit of all-optical links.

Sensitivity to OCS reconfiguration latency. We sweep the emulated latency from 0 to 1000 ms for Configs. 1 and 2 (Figure 10). Both configurations require 6 reconfigurations per training step. We compare against a simple analytical estimate: $T_{\text{native}} + T_{\text{reconfig}} \times N_{\text{reconfig}}$. At 50 ms OCS latency, Opus incurs 1.05 $\times$ and 1.08 $\times$ the native step time for Configs. 1 and 2 respectively. With provisioning, these overheads drop to 1.01 $\times$ and 1.02 $\times$, as speculative reconfiguration hides the OCS delay within the idle window between parallelism phases (§3). The benefit of provisioning is more pronounced in Config. 2, where larger DP groups create wider inter-phase windows due to higher collective synchronization delays.

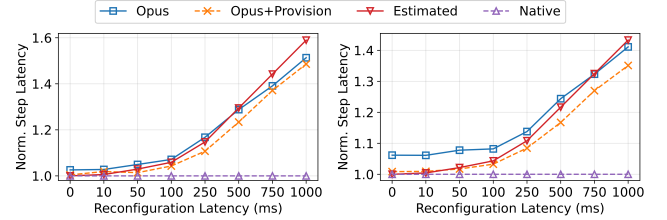


Figure 10: Step latency vs. emulated OCS reconfiguration latency for Config. 1 (left) and Config. 2 (right).

Control-plane overhead. To isolate Opus’s control-plane overhead from OCS reconfiguration delay, we set the emulated latency to 0 ms and measure Config. 2 at 64 GPUs (Figure 11 (left)). The overhead—comprising `pre_comm/post_comm` logic, per-rail locking during phase transitions, and rank synchronization through the controller—is 6.13%. With provisioning, the overhead drops to 0.79%, because the synchronization cost is masked within inter-phase idle windows.

Figure 11 (right) shows Config. 3, which places 4-way PP in the scale-out with no DP. Since all scale-out communication belongs to a single parallelism dimension, Opus’s topology orchestrator correctly suppresses all reconfigurations: the step latency is unchanged between 0 ms and 100 ms emulated OCS latency. The 6.46% overhead is entirely due to Opus’s per-operation control logic for PP, which issues controller requests for every asymmetric Send/Recv (§4.2). Also, the step duration does not increase with increased reconfiguration latency, showing that the Opus topology orchestrator avoids unnecessary network reconfigurations.

5.3 Large-scale Simulation of Opus

We evaluate Opus at large scale using ASTRA-sim [69], a distributed ML system simulator, with Chakra ET [78] traces that replay realistic training iterations under hybrid parallelism. We simulate two Llama/GPT-style dense models (hidden dimension 8192, feed-forward dimension 28672, 64 attention heads, 8 KV heads, 96 layers, sequence length 4096, and global batch size 256) and a DeepSeek-V3-style mixture-of-experts model, and report end-to-end iteration time.

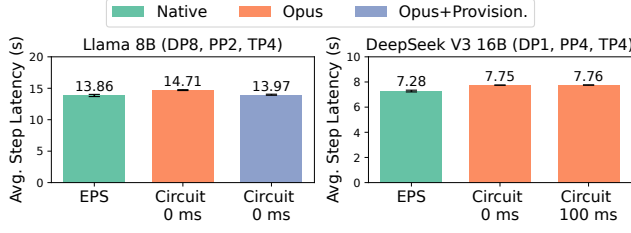


Figure 11: Control-plane overhead. (left) Config. 2 (64 GPUs): Opus vs. Opus +Provisioning vs. native EPS at 0 ms emulated OCS latency. (right) Config. 3: Opus at 0 ms and 100 ms OCS latency vs. native EPS. No reconfigurations occur, confirming Opus suppresses unnecessary topology changes.

Reconfigurable network backend. We extend ASTRA-sim with a *reconfigurable analytical* network backend that supports time-varying connectivity. The backend consumes a set of candidate circuit configurations, each expressed as a directed bandwidth matrix indexed by a topology ID. Zeroes in the matrix represent absent circuits. The active matrix changes dynamically as Opus selects configurations at runtime, while base link latency and reconfiguration latency are applied uniformly from a YAML configuration. To ensure correctness, the backend rejects reconfiguration requests while collectives are in flight or another reconfiguration is pending. Accepted reconfigurations drain active links before applying new settings. During the reconfiguration interval, arriving traffic queues on links and is released upon completion.

Baselines. We compare Opus against two baselines: (1) *EPS*: a static electrical-packet-switched network where all links that Opus could form across any circuit configuration are always active, giving EPS strictly higher total bandwidth; and (2) *Ideal one-shot*: a network reconfigured once before the job starts, with Opus’s total bandwidth divided optimally across parallelism dimensions without port or link granularity constraints, following related work [87]. The EPS baseline does not model congestion in switches.

Effect of OCS reconfiguration latency. We sweep OCS reconfiguration latency from 0 to 1,000 ms for two setups: Llama-80B on 128 H200 GPUs (DP=4, PP=4, TP=8, 400 Gbps scale-out) and GPT-80B on 512 B200 GPUs (DP=4, PP=4, TP=32, 800 Gbps scale-out), shown in Figures 12 and 13. Opus’s overhead remains low at production-relevant OCS latencies. On the H200 cluster at 100 ms OCS latency, Opus with provisioning is 5.31% slower than EPS and 3.32% slower than ideal one-shot. On the B200 cluster at 10 ms latency, the gaps narrow to 2.49% and 0.93% respectively. The GPT model on B200 is more sensitive to reconfiguration latency due to its higher communication intensity.

Effect of scale-out bandwidth. We sweep scale-out bandwidth from 100 Gbps to 1,600 Gbps at 10 ms OCS latency (Figures 12, 13, right panels). Higher bandwidth reduces Opus’s overhead by shrinking the communication time relative to the fixed reconfiguration cost. Compared to ideal one-shot, Opus’s overhead drops from 7.73% to 0.72% on H200 and from 11.63% to 0.34% on B200 as bandwidth increases from 100 Gbps to 1,600 Gbps. We scale GPU count from 64 to 2,048 by increasing the DP degree while holding TP and PP fixed (Figure 15). Opus’s overhead remains stable: at 512 H200 GPUs (10

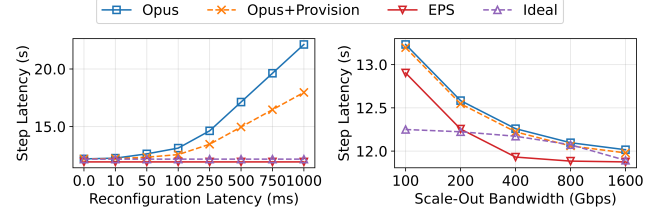


Figure 12: Llama-80B on 128 H200 GPUs (DP=4, PP=4, TP=8). OCS latency sweep at 400 Gbps scale-out (left); bandwidth sweep at 10 ms OCS latency (right).

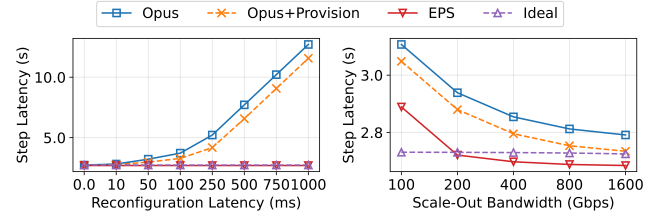


Figure 13: GPT-80B on 512 B200 GPUs (DP=4, PP=4, TP=32). OCS latency sweep at 800 Gbps scale-out (left); bandwidth sweep at 10 ms OCS latency (right).

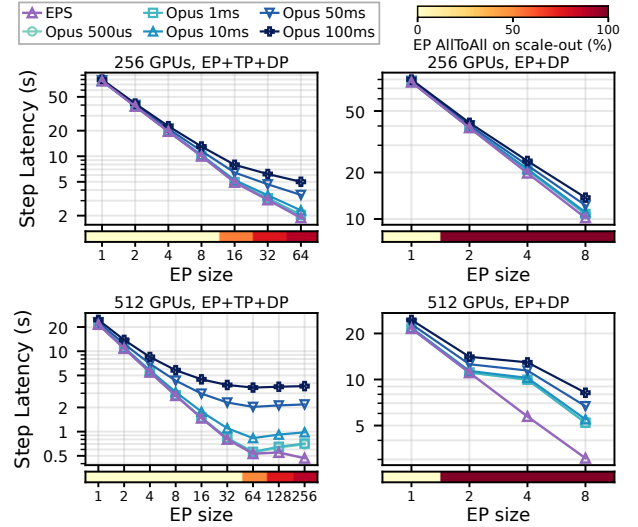


Figure 14: Step latency vs. EP degree for Opus and the EPS baseline. Left: EP co-located with TP/DP. Right: EP spread over DP only, so all EP ALLToALL crosses scale-out.

ms OCS, 400 Gbps), Opus with provisioning is 6.62% slower than EPS. At 2,048 B200 GPUs (10 ms OCS, 800 Gbps), the gap is 11.22%. **Expert-parallelism support.** We compare Opus against EPS by sweeping the fraction of expert-parallel (EP) ALLToALL traffic that crosses the scale-out fabric from 0% to 100% (Figure 14). We simulate DeepSeek-V3-style MoE model training using Chakra ET trace format: 256 routed experts with top-8 gating, sequence length 4096, and a global batch of 256. We use two clusters: a 256×H200 cluster (8×GPU 450 GB/s scale-up, 400 Gbps scale-out, TP=8, PP=4, DP=8) and a 512×H200 cluster (hypothetical 32×GPU 900 GB/s

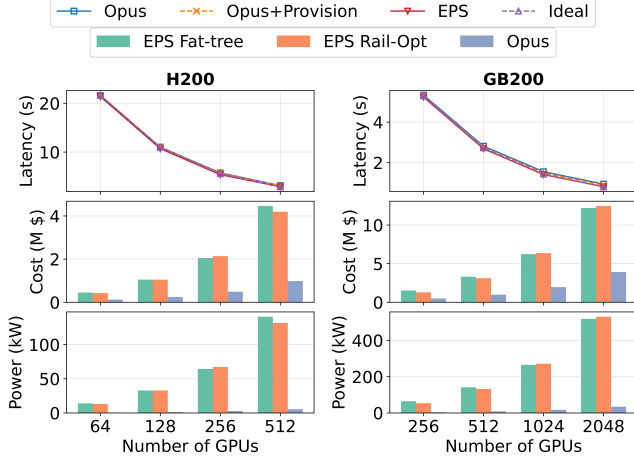


Figure 15: Performance, cost, and power scaling. DGX H200 with 400 Gbps links (left); B200 with 800 Gbps links (right). Cost and power exclude fiber cables [17–19, 43, 51, 63].

scale-up, 800 Gbps scale-out, TP=32, PP=2, DP=8). To fully test Opus performance, we select two expert placement strategies: $EP+TP+DP$, where EP overlaps with TP (small EP groups stay inside a scale-up domain) and spills onto scale-out over DP as EP grows [38, 89], and $EP+DP$, an adversarial case where EP only overlaps with DP groups [68], and the EP ALLToALL is carried entirely by the scale-out fabric (100%). We scale the EP size from the minimal (EP=1) to the maximal (bounded by the pipeline stage size).

When the EP ALLToALL is fully contained by a scale-up domain (0% on scale-out), Opus, with provisioning, matches EPS to within 0.5% with 0.5 ms–1 ms OCS latencies and within 3% at 10 ms. As the EP degree grows and a larger share of the ALLToALL spills onto scale-out, Opus reconfigures to serve it, and the cost of using multi-hop paths appears (Appendix 5D). With placement $EP+TP+DP$ at 256 GPUs, Opus stays within 7.3% of EPS when 88.9% of the EP ALLToALL crosses scale-out (0.5 ms switching), rising to 8.1% at 1 ms and 22.5% at 10 ms. The degradation grows with both the scale-out EP fraction and the OCS latency, and is more significant in the adversarial case (EP + DP). Opus performance degradation is bounded because today’s large-scale MoE training rarely uses an EP size more than 64 [14], and usually EP is overlapped with TP [38, 89].

5.4 Cost and Power

We estimate the networking cost and power of Opus and the two EPS baselines, *fat-tree* and *rail-optimized*, with a per-port accounting model. We consider two clusters, a 400G H200 cluster and an 800G B200 cluster. A cluster has S scale-up domains, each with G GPUs, for SG GPUs in total, where every GPU drives one scale-out NIC port through one optical transceiver. The scale-out fabric model contains three devices—electrical switch ports, OCS ports, and transceivers—each with a fixed unit cost and power (Table 2). The total cost (and, identically, the total power) of a topology is

$$\text{Cost} = n_{\text{sw}} c_{\text{sw}} + n_{\text{ocs}} c_{\text{ocs}} + n_{\text{xcvr}} c_{\text{xcvr}},$$

where n_{sw} , n_{ocs} , and n_{xcvr} are the port counts and $c_{\bullet}$ are the corresponding unit values.

Component	Config	Power (W)	Cost (\$)
Electrical switch port (Tomahawk-4)	H200, 400G	39.4 [17]	583 [17]
Electrical switch port (Quantum-X800 XDR)	B200, 800G	48.6 [51]	1,041 [43]
OCS port (Polatis 6000N)	400G/800G/1.6T	0.39 [63]	520 [63]
Optical transceiver (MMA4Z00-NS)	400G	10 [18]	719 [18]
Optical transceiver (MMA4Z00-NS)	800G	16 [19]	879 [19]
Optical transceiver (MMS4C11)	1.6T	30 [44]	2,099 [44]

Table 2: Per-port cost and power used in calculation. Switch and OCS per-port figures are the device totals divided by port count (64 ports for the H200 switch, 144 for the B200 switch, 192 for the OCS). H200 clusters use the 400G transceiver, B200 clusters the 800G transceiver, and the OCS can be paired with 1.6T transceivers.

The topologies differ in their switch, OCS, and transceiver counts. Every GPU NIC needs one transceiver, so the GPU-side count is SG in all cases. With pluggable optics each electrical switch port also terminates in a transceiver, so a switched fabric needs $n_{\text{xcvr}} = SG + n_{\text{sw}}$ transceivers, which grows with its switch count. Co-packaged-optics (CPO) switches instead integrate the optics into the switch ASIC and add no separate transceiver, leaving $n_{\text{xcvr}} = SG$. The H200 baseline uses electrical switches with pluggable optics, and the B200 baseline uses CPO switches [8]. *EPS fat-tree* is a three-tier folded-Clos network. With switch radix $k = \lfloor (4SG)^{1/3} + 0.5 \rfloor$ it uses $k^2 + k^2/4$ switches ($n_{\text{sw}} = k(k^2 + k^2/4)$ ports). *EPS rail-optimized* builds a leaf-spine fat-tree per rail and has a cluster core layer that interconnects the rails. With switch radix $k = \lfloor (4S)^{1/2} + 0.5 \rfloor$, it uses $2G \lfloor S/(k/2) \rfloor + \lfloor GS/k \rfloor$ switches ($2 \lfloor S/(k/2) \rfloor$ leaf-spine switches per rail, replicated across the G rails, plus $\lfloor GS/k \rfloor$ cluster-core switches). This gives $n_{\text{sw}} = k(2G \lfloor S/(k/2) \rfloor + \lfloor GS/k \rfloor)$ ports. *Opus* replaces the entire electrical fabric with one OCS per rail, so there are G OCSeS of radix $2S$, assuming the OCS ports are uni-directional, and one GPU port (bi-directional) is connected to two OCS ports. This gives $n_{\text{sw}} = 0$, $n_{\text{ocs}} = 2SG$, and $n_{\text{xcvr}} = SG$, as the OCS is optically transparent and adds no transceiver.

Figure 15 shows the savings of Opus based on this cost model in addition to the workload performance. An OCS port draws about two orders of magnitude less power than an electrical switch port (0.39 W vs. 48.6 W at 800G), and Opus removes the entire multi-tier switch fabric, so its residual power comes almost entirely from the GPU transceivers. This yields large power savings of 15.4× on B200 (rail-optimized, 512–2,048 GPUs) and 23.9× on H200 (rail-optimized, 64–512 GPUs) at a small performance degradation. The cost savings are smaller, 3.2× and 4.3×, because those same GPU transceivers (\$879 each at 800G) remain a large part of Opus’s cost. Opus saves more compared to H200 than B200 on both metrics because the H200 pluggable-optics cluster requires a transceiver on every switch port, all of which Opus removes, whereas the B200 CPO cluster carries no switch-side transceiver. As OCSeS are bandwidth-agnostic, Opus can scale to 1.6T transceivers [44], matching B200-800G EPS performance (0.2% slower, Figure 13) at 1.94× lower network cost and 8.42× lower power consumption.

6 Related Work

Optical network designs for general datacenters. Prior work has explored optically reconfigurable network topologies using OCSes in traditional datacenter networks, following either traffic-agnostic [2, 4, 42, 74] or traffic-aware [11, 15, 23, 25, 34, 82] reconfiguration schemes. Traffic-agnostic designs cycle through topologies at fixed intervals, while traffic-aware designs reconfigure based on estimated demand. These schemes generally target one of two cases: microsecond-scale reconfiguration with shallow buffers for latency-sensitive traffic, or second-scale reconfiguration with deep buffers for elephant flows. Neither regime fits distributed ML workloads, which require both low-latency connectivity and sustained high bandwidth across hundreds of GPUs in a collective communication group. This is the gap Opus addresses.

Reconfigurable optical networks for ML. A separate line of work reconfigures optical topologies before a job starts, exploiting ML traffic predictability to allocate bandwidth across parallelism dimensions [29, 30, 36, 81, 84, 88, 93]. However, these designs limit the number of supported parallelisms to the per-accelerator port count—for example, a TPU torus cannot efficiently support more than three parallelism dimensions. Increasing port count strains server architectures or requires technology not yet commercially available [88]. Recent work on in-job reconfiguration [87] alleviates but does not eliminate this bottleneck.

Opus and rail-optimized fabrics. In contrast, Opus operates within the widely deployed rail-optimized topology [22, 60, 67, 83] and requires only 2-degree scale-out connectivity to form reconfigurable rings—well within the capabilities of existing server NICs and commercially available OCSes. By time-multiplexing circuits across parallelism phases rather than provisioning static per-dimension links, Opus decouples the number of supported parallelisms from the per-GPU port count entirely. To our knowledge, Opus is the first system to demonstrate end-to-end ML training on photonic rails, validated on physical hardware, emulated at medium scale, and simulated at up to 2,048 GPUs.

7 Discussion

Generality of Opus. Opus supports up to 10 parallelism dimensions via its topology encoding (§4), easily accommodating the 3–5 dimensions typical of today’s large-scale LLM pre-training [12, 14]. Beyond supervised training, Opus can serve any ML workload whose communication can be decomposed into predictable phases. For instance, OCS reconfiguration schedules can be tied to the KV-cache transfer pipelines in prefill-decode disaggregated inference workloads [62, 92]. Similarly, Opus can serve RL post-training workloads which alternate between inference and training with distinct parallelism requirements [85]. For multi-job environments, Opus composes per-job topologies into a global OCS configuration, and the non-blocking properties of commodity OCSes [64] ensure that reconfiguring one job’s circuits does not disrupt another’s. Finally, for ALLToALL traffic in expert parallelism, additional scheduling logic can be implemented to provision optimized topologies through demand prediction [36], with its benefit limited by the predictability of the future expert ALLToALL traffic and the prediction accuracy.

Scalability. Opus scales to large clusters using commodity OCSes. Liquid-crystal OCSes offer up to 512 ports [13], which suffices to connect large scale-up domains (e.g., NVL72) at hyperscaler-relevant scale—up to 18K GPUs per rail. Beyond this, hierarchical or multi-plane topologies such as BCube [3, 86, 87] can extend reach further, though current server and NIC architectures limit the number of per-GPU ports available to support additional hierarchies or network planes. The ever-increasing switching speed of the optical interconnect (switches and transceivers) can also help address the diminishing window sizes at large scales (§ 3.2).

Fault tolerance. Photonic rails inherit the fault model of rail-optimized topologies [83] without introducing new failure modes, while reducing failure points by eliminating switch ASICs, transceivers, and lasers from the datapath. Passive optical technologies [4, 5] and rail redundancy can improve reliability further. Opus is compatible with existing software-layer fault handling [20, 28, 33] and standard SDN controller replication [6, 27].

8 Conclusion

In this work, we propose a novel photonic rail topology for distributed ML training, which offers comparable training performance to state-of-the-art electrical networks but achieves 4× reduction in networking infrastructure cost and over 23× reduction in network power consumption using existing optical-circuit-switching technologies. To realize the vision, we develop a complete control plane, Opus, that orchestrates topology reconfigurations based on the application’s communication intents. We deploy the system on a real optical-circuit-switched testbed, evaluate the system performance on the Perlmutter supercomputer, and demonstrate the scalability of its performance and efficiency to 2,000+ GPUs in simulation.

Ethics: This work does not raise any ethical issues.

Acknowledgements

We sincerely thank the anonymous SIGCOMM reviewers for their insightful feedback. This work was supported in part by ACE, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA. The authors of this work are also supported by NSF Awards No. 2444537 and No. 2435852. This research used resources of the National Energy Research Scientific Computing Center, a DOE Office of Science User Facility supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231 using NERSC award ASCR-ERCAP0038386.

References

- [1] Saksham Agarwal, Qizhe Cai, Rachit Agarwal, David Shmoys, and Amin Vahdat. 2024. Harmony: A congestion-free datacenter architecture. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 329–343.
- [2] Daniel Amir, Nitika Saran, Tegan Wilson, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. 2024. Shale: A practical, scalable oblivious reconfigurable network. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 449–464.
- [3] Joao Araujo, Alex Chow, Mark Handley, Ryder Lewis, Christoph Paasch, Jitendra Padhye, Michael Papamichael, Greg Steinbrecher, Amin Tootoonchian, Lihua Yuan, S. Anantharamu, Abhishek Dosi, Mohit Garg, Mahdih Ghazi, Torsten Hoefler, Deepal Jayasinghe, Jithin Jose, Abdul Kabbani, Guohan Lu, Yang Wang, K. Doddapaneni, Murali Garimella, Vipin Jain, Yanfang Le, H. Nagulapalli, S. Narayanan, Rong Pan, Rathina Sabesan, Raghava Sivaramu, Rip Sohan, Eric Davis, Dragos Dumitrescu, Mohan Kalkunte, Bhaswar Mitra, Guglielmo Morandin, Adrian Popa, Costin Raiciu, Eric Spada, John Spillane, Niranjan Vaidya, Aviv Barnea, Idan Burstein, Elazar Cohen, Yamin Friedman, Noam Katz, Masoud Moshref, Yuval Shpigelman, Shahaf Shuler, Shy Shyman, and Sayantan Sur.

2026. Resilient AI Supercomputer Networking using MRC and SRv6. (2026). arXiv:cs.NI/2605.04333 <https://arxiv.org/abs/2605.04333>
- [4] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, Benn Thomsen, and Hugh Williams. 2020. Sirius: A Flat Datacenter Network with Nanosecond Optical Switching. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 782–797. <https://doi.org/10.1145/3387514.3406221>
 - [5] Kaoutar Benyahya, Ariel Gomez Diaz, Junyi Liu, Vassily Lyutsarev, Marianna Pantouvak, Kai Shi, Shawn Yohanes Siew, Hitesh Ballani, Thomas Burrridge, Daniel Cletheroe, Thomas Karagiannis, Brian Robertson, Ant Rowstron, Mengyang Yang, and Paolo Costa. 2025. Mosaic: Breaking the Optics versus Copper Trade-off with a Wide-and-Slow Architecture and MicroLEDs. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 234–247. <https://doi.org/10.1145/3718958.3750510>
 - [6] Pankaj Berde, Matteo Gerola, Jonathan Hart, Yuta Higuchi, Masayoshi Kobayashi, Toshio Koide, Bob Lantz, Brian O'Connor, Pavlin Radoslavov, William Snow, and Guru Parulkar. 2014. ONOS: towards an open, distributed SDN OS. In *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking (HotSDN '14)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/2620728.2620744>
 - [7] Maciej Besta and Torsten Hoefler. 2014. Slim Fly: A Cost Effective Low-Diameter Network Topology. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 348–359. <https://doi.org/10.1109/SC.2014.34>
 - [8] Broadcom Inc. 2025. BCM78909 51.2-Tb/s Multilayer Co-Packaged Optics Switch. Online; accessed July 5, 2025. (2025). <https://www.broadcom.com/products/fiber-optic-modules-components/co-packaged-optics/switches/bcm78909> A high-radix, high-bandwidth CPO switch supporting up to 64×800GbE or 128×400GbE.
 - [9] Broadcom Inc. 2025. Co-Packaged Optics (CPO). <https://www.broadcom.com/info/optics/cpo>. (2025). Accessed: 2025-07-03.
 - [10] Broadcom Inc. Optical Systems Division. 2021. *SiPh Chiplets In Package (SCIP)*. Technical Report. Broadcom Inc., Irvine, CA, USA. <https://docs.broadcom.com/doc/siph-chiplets-in-package-scip> OSD CPO SCIP_20211106 V5.
 - [11] Li Chen, Kai Chen, Zhonghua Zhu, Minlan Yu, George Porter, Chunming Qiao, and Shan Zhang. 2017. Enabling Wide-Spread Communications on Optical Fabric with MegaSwitch. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 577–593.
 - [12] Weiwei Chu, Xinfeng Xie, Jiecao Yu, Jie Wang, Amar Phanishayee, Chunqiang Tang, Yuchen Hao, Jianyu Huang, Mustafa Ozdal, Jun Wang, Vedanuj Goswami, Naman Goyal, Abhishek Kadian, Andrew Gu, Chris Cai, Feng Tian, Xiaodong Wang, Min Si, Pavan Balaji, Ching-Hsiang Chu, and Jongsoo Park. 2025. Scaling Llama 3 Training with Efficient Parallelism Strategies. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1703–1716. <https://doi.org/10.1145/3695053.3731410>
 - [13] Coherent Corp. 2025. Optical Circuit Switch (OCS). <https://www.coherent.com/networking/optical-circuit-switch>. (2025). Accessed: 2025-07-10; Based on press release published March 25, 2024; Coherent's liquid-crystal-based OCS architecture supports up to 300×300 ports and is optimized for AI/ML data center fabrics.
 - [14] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qishu Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shutong Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. (2025). arXiv:cs.CL/2412.19437 <https://arxiv.org/abs/2412.19437>
 - [15] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2010. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 339–350.
 - [16] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
 - [17] FS.COM. n.d. N9510-64D 64-Port Ethernet L3 Data Center Switch (Broadcom Tomahawk-4, 64×400GbE). <https://www.fs.com/products/149853.html>. (n.d.). Accessed: 2025-07-02.
 - [18] FS.COM. n.d. NVIDIA/Mellanox MMA4Z00-NS400 Compatible 400GBASE-SR4 OSFP RHS/Flat Top 4×100G PAM4 850nm 50m MPO-12/APC MMF Ethernet Transceiver Module. https://www.fs.com/products/310009.html?now_cid=3652. (n.d.). Accessed: 2026-06-20.
 - [19] FS.com Inc. 2025. NVIDIA/Mellanox MMA4Z00-NS Optical Transceiver Module. <https://www.fs.com/products/229253.html>. (2025). Product page, Accessed: 2026-02-06.
 - [20] Swapnil Gandhi, Mark Zhao, Athinagoras Skiadopoulos, and Christos Kozyrakis. 2024. Recycle: Resilient training of large DNNs using pipeline adaptation. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 211–228.
 - [21] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Rifadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 57–70. <https://doi.org/10.1145/3651890.3672233>
 - [22] Alexandru M Gherghescu, Vlad-Andrei Bădoi, Alexandru Agache, Mihai-Valentin Dumitru, Iuliu Vasilescu, Radu Mantu, and Costin Raiciu. 2024. I've Got 99 Problems But FLOPS Ain't One. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*. 195–204.
 - [23] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireeja Ranade, Pierre-Alexandre Blanche, Houman Rastegarfar, Madeleine Glick, and Daniel Kilper. 2016. ProjecToR: Agile Reconfigurable Data Center Interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16)*. Association for Computing Machinery, New York, NY, USA, 216–229. <https://doi.org/10.1145/2934872.2934911>
 - [24] Albert Greenberg, James R. Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A. Maltz, Parveen Patel, and Sudipta Sengupta. 2009. VL2: A Scalable and Flexible Data Center Network. In *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication (SIGCOMM '09)*. Association for Computing Machinery, New York, NY, USA, 51–62. <https://doi.org/10.1145/1592568.1592576>
 - [25] Navid Hamedazimi, Zafar Qazi, Himanshu Gupta, Vyas Sekar, Samir R. Das, Jon P. Longtin, Himanshu Shah, and Ashish Tanwer. 2014. FireFly: A Reconfigurable Wireless Data Center Fabric Using Free-Space Optics. In *Proceedings of the 2014 ACM Conference on SIGCOMM (SIGCOMM '14)*. Association for Computing Machinery, New York, NY, USA, 319–330. <https://doi.org/10.1145/2619239.2626328>
 - [26] Vipul Harsh, Sangeetha Abdu Jyothi, and P. Brighten Godfrey. 2020. Spineless data centers. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks*. 67–73.
 - [27] Patrick Hunt, Mahadev Konar, Flavio P Junqueira, and Benjamin Reed. 2010. ZooKeeper: Wait-free coordination for internet-scale systems. In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*.
 - [28] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. 2023. Ooblock: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 382–395.
 - [29] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. <https://doi.org/10.1145/3579371.3589350>
 - [30] Mehrdad Khani, Manya Ghobadi, Mohammad Alizadeh, Ziyi Zhu, Madeleine Glick, Keren Bergman, Amin Vahdat, Benjamin Klenk, and Eiman Ebrahimi. 2021. SiP-ML: high-bandwidth optical network interconnects for machine learning

- training. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 657–675.
- [31] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.
 - [32] Abhishek Vijaya Kumar, Arjun Devraj, Darius Bunandar, and Rachee Singh. 2024. A case for server-scale photonic connectivity. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. Association for Computing Machinery, New York, NY, USA, 290–299. <https://doi.org/10.1145/3696348.3696856>
 - [33] ChonLam Lao, Jiaqi Gao, Jiamin Cao, Zhipeng Zhang, Pengcheng Zhang, Jiangfei Duan, Zhilong Zheng, Yu Guan, Yichi Xu, Yong Li, Zhengping Qian, Aditya Akella, Minlan Yu, Ennan Zhai, Dennis Cai, and Jingren Zhou. 2026. TrainMover: An Interruption-Resilient Runtime for ML Training. (2026). [arXiv:cs.DC/2412.12636](https://arxiv.org/abs/2412.12636) <https://arxiv.org/abs/2412.12636>
 - [34] Cong Liang, Xiangli Song, Jing Cheng, Mowei Wang, Yashe Liu, Zhenhua Liu, Shizhen Zhao, and Yong Cui. 2024. NegotiaToR: Towards A Simple Yet Effective On-demand Reconfigurable Datacenter Network. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 415–432.
 - [35] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, Sanket Purandare, Gokul Nadathur, and Stratos Idreos. 2025. TorchTitan: One-stop PyTorch native solution for production ready LLM pre-training. (2025). [arXiv:cs.CL/2410.06511](https://arxiv.org/abs/2410.06511) <https://arxiv.org/abs/2410.06511>
 - [36] Xudong Liao, Yijun Sun, Han Tian, Xinchun Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, Zhizhen Zhong, Guyue Liu, Ying Zhang, Xiaofeng Ye, Yiming Zhang, and Kai Chen. 2025. MixNet: A Runtime Reconfigurable Optical-Electrical Fabric for Distributed Mixture-of-Experts Training. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 554–574. <https://doi.org/10.1145/3718958.3750465>
 - [37] Linux. [n. d.]. *ethtool(8) - Linux man page*. <https://linux.die.net/man/8/ethtool>
 - [38] Dennis Liu, Zijie Yan, Xin Yao, Tong Liu, Vijay Korthikanti, Evan Wu, Shiqing Fan, Gao Deng, Hongxiao Bai, Jianbin Chang, Ashwath Aithal, Michael Andersch, Mohammad Shoeybi, Jiajie Yao, Chandler Zhou, David Wu, Xipeng Li, and June Yang. 2026. MoE Parallel Folding: Heterogeneous Parallelism Mappings for Efficient Large-Scale MoE Model Training with Megatron Core. (2026). [arXiv:cs.LG/2504.14960](https://arxiv.org/abs/2504.14960) <https://arxiv.org/abs/2504.14960>
 - [39] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).
 - [40] Lumentum Holdings Inc. 2025. Lumentum Optical Circuit Switch to Improve Next-Generation AI Data Center Scalability. <https://www.lumentum.com/en/media-room/news-releases/lumentum-optical-circuit-switch-improve-next-generation-ai-data-center>. (26 March 2025). Accessed June 20, 2025.
 - [41] Changhai Man, Joongun Park, Hanjiang Wu, Huan Xu, Srinivas Sridharan, and Tushar Krishna. 2025. STAGE: A Symbolic Tensor grAph GEnerator for distributed AI system co-design. *arXiv preprint arXiv:2511.10480* (2025).
 - [42] William M Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C Snoeren, and George Porter. 2017. RotorNet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 267–280.
 - [43] NADDOD. 2025. NVIDIA Quantum-X800 XDR InfiniBand Switch, Q3400-RA. <https://www.naddod.com/products/nvidia-networking/102612>. (2025). Reseller price listing, Accessed: 2026-02-06.
 - [44] NADDOD. 2025. NVIDIA Mellanox MMS4C11-XM-RHS Compatible 1.6T 2xDR4 OSFP Silicon Photonics Optical Transceiver Module. <https://www.naddod.com/products/103418.html>. (2025). Product page, Accessed: 2026-06-19.
 - [45] National Energy Research Scientific Computing Center (NERSC). 2025. Perlmutter Architecture — NERSC Documentation. <https://docs.nersc.gov/systems/perlmutter/architecture/>. (2025). Accessed: 2025-07-04.
 - [46] nEye Systems. 2025. nEye: Dismantling Network Walls to Build a Sustainable AI Future. <https://www.neye.ai/>. (2025). Optical circuit switch platform for AI datacenter networking. Accessed: 2025-02-02.
 - [47] NVIDIA. 2024. *NVIDIA Firmware Tools (MFT) Documentation*. <https://docs.nvidia.com/networking/display/nvidia-firmware-tools-mft-documentation-v4-32-0.0.pdf>
 - [48] NVIDIA Corporation. 2020. *NVIDIA Collective Communication Library (NCCL): Creating a Communicator*. NVIDIA. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/communicators.html> Accessed July 6, 2025.
 - [49] NVIDIA Corporation. 2022. Doubling all-to-all Performance with NCCL 2.12: Introducing PXN (PCI X NVLink). NVIDIA Developer Blog. (Feb. 2022). <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/> Describes PXN, which enables GPU-to-NIC communication via NVLink to optimize rail-aligned collective performance.
 - [50] NVIDIA Corporation. 2024. ConnectX-7 400G Adapters Datasheet. <https://resources.nvidia.com/en-us-accelerated-networking-resource-library/connectx-7-datasheet>. (2024). Accessed: 2025-07-02.
 - [51] NVIDIA Corporation. 2024. NVIDIA Q32xx and Q34xx XDR 800Gb/s InfiniBand Switch Systems User Manual. <https://docs.nvidia.com/networking/display/xdrswitcheshwum/specifications>. (2024). Accessed: 2026-02-06.
 - [52] NVIDIA Corporation. 2025. ConnectX-6 Dx Firmware Download. <https://network.nvidia.com/support/firmware/connectx6dx/>. (2025). Accessed: 2026-02-06.
 - [53] NVIDIA Corporation. 2025. Co-Packaged Silicon Photonics Networking Switches. Online; accessed July 5, 2025. (2025). <https://www.nvidia.com/en-us/networking/products/silicon-photonics/> Describes NVIDIA's co-packaged optics (CPO) switches with integrated silicon photonics.
 - [54] NVIDIA Corporation. 2025. *NVIDIA Announces Spectrum-X Photonics, Co-Packaged Optics Networking Switches to Scale AI Factories to Millions of GPUs*. Press Release. NVIDIA Corporation, Santa Clara, CA, USA. <https://nvidianews.nvidia.com/news/nvidia-spectrum-x-co-packaged-optics-networking-switches-ai-factories> Unveiled at GTC 2025.
 - [55] NVIDIA Corporation. 2025. *NVIDIA Collective Communications Library (NCCL)*. NVIDIA Developer. <https://developer.nvidia.com/nccl> Version 2.x; MPI-compatible multi-GPU / multi-node collective communication library.
 - [56] NVIDIA Corporation. 2025. *NVIDIA DGX H200 Datasheet*. Datasheet. NVIDIA Corporation, Santa Clara, CA. <https://resources.nvidia.com/en-us-dgx-systems/dgx-h200-datasheet> Includes specifications of the DGX H200 system, featuring 8x H200 GPUs, dual Xeon Platinum 8480C CPUs, 2 TB system memory, 30 TB NVMe SSD, and full NVIDIA AI Enterprise software stack.
 - [57] NVIDIA Corporation. 2025. *NVIDIA DGX SuperPOD*. NVIDIA. <https://www.nvidia.com/en-us/data-center/dgx-superpod/> Full-stack data center platform scaling to tens of thousands of GPUs; includes compute, networking, storage, and software.
 - [58] NVIDIA Corporation. 2025. NVIDIA Ethernet Driver for Linux (mlnx_en). https://network.nvidia.com/products/ethernet-drivers/linux/mlnx_en/. (2025). Accessed: 2026-02-06.
 - [59] NVIDIA Corporation. 2025. *NVIDIA HGX Platform*. NVIDIA. <https://www.nvidia.com/en-us/data-center/hgx/> Reference architecture combining GPUs, NVLink/NVSwitch, networking, and AI/HPC software stack.
 - [60] NVIDIA Corporation. 2025. *Rail Optimized Topology Validation*. NVIDIA Networking, Santa Clara, CA. <https://docs.nvidia.com/networking/display/ibdiagnetusermanualv221/Rail+Optimized+Topology+Validation> Part of the ibdiagnet InfiniBand Fabric Diagnostic Tool User Manual; describes cabling validation and compute-fabric alignment in DGX SuperPOD rail-optimized fabrics.
 - [61] Jeremie Eliahou Ontiveros, Dylan Patel, and Wei Zhou. 2025. xAI's Colossus 2 - First Gigawatt Datacenter In The World, Unique RL Methodology, Capital Raise. SemiAnalysis. (Sept. 2025). <https://newsletter.semanalysis.com/p/xais-colossus-2-first-gigawatt-datacenter> Accessed: 2026-01-23.
 - [62] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.
 - [63] Polatis. 2023. Polatis Series 6000n Optical Switch Datasheet. https://www.redhelix.com/wp-content/uploads/2023/11/Polatis_6000n_Data_Sheet-rhl.pdf. (2023). Datasheet, Accessed: 2026-02-06.
 - [64] Polatis (a HUBER+SUHNER company). n.d.. Series 7000 - 384x384-port Software-Defined Optical Circuit Switch. <https://www.polatis.com/series-7000-384x384-port-software-controlled-optical-circuit-switch-sdn-enabled>. asp. (n.d.). Accessed: 2025-07-01.
 - [65] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, and Amin Vahdat. 2022. Jupiter Evolving: Transforming Google's Datacenter Network via Optical Circuit Switches and Software-Defined Networking. In *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22)*. 66–85.
 - [66] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2023. Zero bubble pipeline parallelism. *arXiv preprint arXiv:2401.10241* (2023).
 - [67] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Urata, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 691–706. <https://doi.org/10.1145/3651890.3672265>
 - [68] Samyann Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Kasey, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*.
 - [69] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. AUSTRA-SIM: Enabling SW/HW Co-Design Exploration for Distributed DL

- Training Platforms. In *IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2020, Boston, MA, USA, August 22-26, 2020*. IEEE.
- [70] Jeff Rasley, Sanyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.
- [71] Peter Sanders, Jochen Speck, and Jesper Larsson Träff. 2009. Two-tree algorithms for full bandwidth broadcast, reduction and scan. *Parallel Comput.* 35, 12 (2009), 581–594.
- [72] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [73] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [74] Vishal Shrivastav, Asaf Valadarsky, Hitesh Ballani, Paolo Costa, Ki Suh Lee, Han Wang, Rachit Agarwal, and Hakim Weatherspoon. 2019. Shoal: A Network Architecture for Disaggregated Racks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, 255–270. <https://www.usenix.org/conference/nsdi19/presentation/shrivastav>
- [75] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. 2015. Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google’s Datacenter Network. *SIGCOMM Comput. Commun. Rev.* 45, 4 (aug 2015), 183–197. <https://doi.org/10.1145/2829988.2787508>
- [76] Ankit Singla, P Brighten Godfrey, and Alexandra Kolla. 2014. High throughput data center topology design. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. 29–41.
- [77] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P. Brighten Godfrey. 2012. Jellyfish: Networking Data Centers Randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. USENIX Association, San Jose, CA, 225–238. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/singla>
- [78] Srinivas Sridharan, Taekyung Heo, Louis Feng, Zhaodong Wang, Matt Bergeron, Wenyin Fu, Shengbao Zheng, Brian Coutinho, Saeed Rashidi, Changhai Man, and Tushar Krishna. 2023. Chakra: Advancing Performance Benchmarking and Co-design using Standardized Execution Traces. *arXiv preprint arXiv:2305.14516* (2023).
- [79] Nouamane Tazi, Ferdinand Mom, Haojun Zhao, Phuc Nguyen, Mohamed Mekki, Leandro Werra, and Thomas Wolf. 2025. The Ultra-Scale Playbook: Training LLMs on GPU Clusters. <https://huggingface.co/spaces/nanotron/ultrascale-playbook>. (2025). Accessed: 2025-05-16.
- [80] Rajeev Thakur and William D Gropp. 2003. Improving the performance of collective operations in MPICH. In *European Parallel Virtual Machine/Message Passing Interface Users’ Group Meeting*. Springer, 257–267.
- [81] Abhishek Vijaya Kumar, Eric Ding, Arjun Devraj, Darius Bunandar, and Rachee Singh. 2026. Reconfigurable Torus Fabrics for Multi-tenant ML. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’26)*. Association for Computing Machinery, New York, NY, USA, 1547–1565. <https://doi.org/10.1145/3779212.3790238>
- [82] Guohui Wang, David G. Andersen, Michael Kaminsky, Konstantina Papagiannaki, T.S. Eugene Ng, Michael Kozuch, and Michael Ryan. 2010. C-Through: Part-Time Optics in Data Centers. In *Proceedings of the ACM SIGCOMM 2010 Conference (SIGCOMM ’10)*. Association for Computing Machinery, New York, NY, USA, 327–338. <https://doi.org/10.1145/1851182.1851222>
- [83] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. 2024. Rail-only: A low-cost high-performance network for training LLMs with trillion parameters. In *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 1–10.
- [84] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. 2023. TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 739–767.
- [85] Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. *EuroSys 2025 (30/03/2025-03/04/2025, Rotterdam)* (2025).
- [86] Zhenguo Wu, Liang Yuan Dai, Ziyi Zhu, Asher Novick, Madeleine Glick, and Keren Bergman. 2023. SiP Architecture for Accelerating Collective Communication in Distributed Deep Learning. In *2023 Optical Fiber Communications Conference and Exhibition (OFC)*. 1–3. <https://doi.org/10.1364/OFC.2023.W1G.1>
- [87] Zhenguo Wu, Benjamin Klenk, Larry Dennison, and Keren Bergman. 2025. ACTINA: Adapting Circuit-Switching Techniques for AI Networking Architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1211–1222.
- [88] Zhenguo Wu, Liang Yuan Dai, Yuyang Wang, Songli Wang, and Keren Bergman. 2024. Flexible silicon photonic architecture for accelerating distributed deep learning. *Journal of Optical Communications and Networking* 16, 2 (2024), A157–A168.
- [89] Zijie Yan, Hongxiao Bai, Xin Yao, Dennis Liu, Tong Liu, Hongbin Liu, Ping-tian Li, Evan Wu, Shiqing Fan, Li Tao, Robin Zhang, Yuzhong Wang, Shifang Xu, Jack Chang, Xuwen Chen, Kunlun Li, Yan Bai, Gao Deng, Nan Zheng, Vijay Anand Korthikanti, Abhinav Khattar, Ethan He, Soham Govande, Sangkug Lym, Zhongbo Zhu, Qi Zhang, Haochen Yuan, Xiaowei Ren, Deyu Fu, Tailai Ma, Shunkang Zhang, Jiang Shao, Ray Wang, Vasudevan Rengasamy, Rachit Garg, Santosh Bhavani, Xipeng Li, Chandler Zhou, David Wu, Yingcan Wei, Ashwath Aithal, Michael Andersch, Mohammad Shoeybi, Jiajie Yao, and June Yang. 2026. Scalable Training of Mixture-of-Experts Models with Megatron Core. (2026). [arXiv:cs.DC/2603.07685](https://arxiv.org/abs/2603.07685) <https://arxiv.org/abs/2603.07685>
- [90] Sharada Yeluri. 2023. Optimizing Power Consumption in High-End Routers. <https://www.linkedin.com/posts/sharada-yeluri-sharadayeluriblogs-junipernetworks-routers-activity>. (22 June 2023).
- [91] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. (2023). [arXiv:cs.DC/2304.11277](https://arxiv.org/abs/2304.11277) <https://arxiv.org/abs/2304.11277>
- [92] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*.
- [93] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. 2024. Resiliency at Scale: Managing Google’s TPUv4 Machine Learning Supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 761–774.

Appendix

Appendices are supporting material that has not been peer-reviewed.

A Characteristics of different parallelism strategies

ML systems leverage multiple, co-existing parallelisms (Table 3), with each parallelism type shown in Table 4. These parallelisms include data parallelism (DP and FSDP), pipeline parallelism (PP), tensor parallelism (TP, often with sequence parallelism or SP), context parallelism (CP), and expert parallelism (EP). Each parallelism incurs communication that differs in: data volume, start time, frequency, and communication pattern.

Model size	Compute (N GPUs)	Practices
Small ($< 10B$)	$N \leq 8$	TP or DP
Large ($> 10B$)	$8 < N \leq 512$	TP & PP, TP & DP, or DP
Large ($> 10B$)	$512 < N \leq 1024$	DP & PP, or DP & TP
Large ($> 10B$)	$N > 1024$	TP, DP & PP

Table 3: Rule-of-thumb LLM parallelism strategies [79].

B Additional Reconfiguration Window Study at Frontier Scale

Setup. The frontier-scale characterization in Figure 4(d) and Figure 5 uses a large global batch (15,360). Since frontier model training often uses a varying global batch size in different training phases [14], we repeat the identical strong-scaling sweep (GPT-style [41], PP=4, TP=32, sequence length 4096, 96 layers, B200, ASTRA-sim [69]) at a $5\times$ smaller global batch of 3,072, sweeping GPU count from 2,048 to 16,384. We use the same analytical pipeline as in § 3.2.

Windows shrink with the batch. Table 5 reports the window distribution at each scale. Reducing the global batch also reduces the window sizes relative to the 15,360 case: the median (p50) window at 2,048 GPUs falls from 187 ms to 37.9 ms, and from 23.9 ms to 5.2 ms at 16,384 GPUs. The distribution stays wide—the shortest windows (p5) range from 9.1 ms at 2,048 GPUs down to 0.8 ms at 16,384 GPUs, while the longest (p95) remain in the hundreds of ms.

Utilization vs. reconfiguration latency. Figure 16 sweeps the OCS reconfiguration latency $T_{reconfig}$ and reports the GPU utilization. Utilization is flat while $T_{reconfig}$ stays below most of the windows ($T_{reconfig} \leq 10$ ms) and then degrades if the latency is large. At 16,384 GPUs a 100 ms reconfiguration reduces utilization from 41.3% to 25.9%, whereas a sub-10 ms OCS largely maintains it (39.6%). The results are consistent with the results for larger global batch size in § 3.2, showing that with fast (sub-10 ms) OCS switching, reconfiguration is hidden inside the natural inter-phase windows, even when a smaller batch makes those windows shorter.

C Opus Pseudocode

We provide complete pseudocode for the shim layer (Algorithm 3).

D Rail Topologies for Expert-Parallel ALLToALL

Setup. A rail connects the GPUs of the same local rank across scale-up domains, so a domain with G GPUs has G rails, each driven by its own OCS. An EP group of size E is spread over $m = E/G$ domains (assuming overlapping EP with TP), and its dispatch and combine ALLToALL must move data between every pair of these

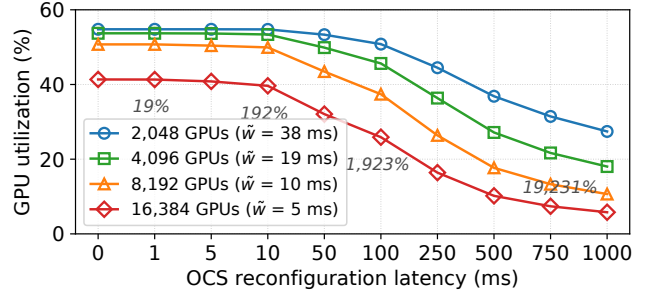


Figure 16: GPU utilization vs. OCS reconfiguration latency for a GPT-style workload on B200 at a smaller global batch of 3,072 (the same setting as in Table 5). $\tilde{w}$ is a workload’s median window. Percentages in the plot show the reconfiguration latency as a fraction of the median window at the largest (16,384-GPU) scale.

m domains. The default Opus connects all domains with the same ring topology across rails. On one ring, two domains can be up to $\lfloor m/2 \rfloor$ hops apart, so their traffic is forwarded through intermediate domains, wasting bandwidth (a bandwidth tax).

Diverging rings across rails. To handle the large fan-out of ALLToALL, Opus instead gives each rail a *different* ring, so different rails connect different pairs of domains. This trades scale-out hops for forwarding inside a domain, which is cheap because the scale-up links have high bandwidth. Each ring connects m pairs directly (every domain to its two ring neighbors), so if a new rail’s ring avoids the pairs already covered, it adds m new direct pairs and gives each domain two more direct neighbors. There are $m(m-1)/2$ unique pairs in all, so G rails connect at most $\min(Gm, m(m-1)/2)$ of them directly, and the fraction of pairs reachable in a single scale-out hop is

$$f(G, m) \leq \min\left(1, \frac{Gm}{m(m-1)/2}\right) = \min\left(1, \frac{2G}{m-1}\right).$$

Equality holds when the rings do not overlap. As long as $G \geq \lceil (m-1)/2 \rceil$ or $m \leq 2G+1$ ($E \leq G(2G+1)$), every pair is directly connected ($f=1$) through Opus with diverged rail topologies. With an NVL72 domain ($G=72$), for instance, this holds for EP groups of up to $G(2G+1) = 10,440$ GPUs, well above typical sizes ($E \leq 128$).

Ring assignment. A ring is an ordering π of the domains, and the pairs it connects are the neighbors in that ordering, $E(\pi) = \{\{\pi_k, \pi_{(k+1)\%m}\} : 0 \leq k < m\}$. Let $w(e)$ be the number of rails that already connect pair e . Algorithm 4 iteratively determines the rail ordering using a greedy method: for each rail it picks the ring that adds the most new pairs, $u(\pi) = |\{e \in E(\pi) : w(e) = 0\}|$. If there are ties, we select the ring whose pairs have been used the least so far ($\min \max_{e \in E(\pi)} w(e)$), which spreads any repeats evenly.

Routing. Opus control router sends each SEND/RCV pair in ALLToALL over a rail that connects its two domains directly, splitting the load across all available rails. If there is an uncovered pair (large m , small G), the router selects the shortest multi-hop path instead, prioritizing forwarding in scale-up domains over scale-out. Since almost all pairs are direct, this extra forwarding is negligible (§5.3). The diverged topologies across rails are set up before the dispatch ALLToALL, overlapping with the expert-router computation (§4.2). **Simulating multi-hop forwarding.** Our reconfigurable network backend (§5.3) models the forwarding cost of these routes directly.

Parallelism	Memory reduction	Compute reduction	Communication type and frequency	Symmetrical?
DP	gbs/dp	gbs/dp	bwd AR per layer/per model	yes
FSDP	gbs/dp, params/dp	gbs/dp	fwd AG, bwd RS per layer/model	yes
TP	params/tp, grads/tp, optims/tp	params/tp	fwd bwd AR per operator	yes
TP & SP	params/tp, grads/tp, optims/tp, actives/tp	params/tp, actives/tp	fwd bwd AG&RS per operator	yes
CP	kv_cache/cp, seq/cp	seq/cp	fwd AG bwd RS per layer	yes
PP	params/pp, grads/pp, optims/pp, actives/pp	params/pp	fwd bwd SEND/RECV per microbatch	no
EP	experts/ep	experts/ep	fwd bwd ALLToALL per layer	yes

Table 4: Characteristics of different parallelism strategies [35]. gbs: global batch size. dp: data parallel degree. seq: sequence length. fwd: forward pass. bwd: backward pass. AR: ALLREDUCE. AG: ALLGATHER. RS: REDUCESCATTER. params: model parameter size. grads: gradients size. optims: optimizer states. actives: activation states.

Algorithm 3 Communication Pattern Scheduling with Topology Reconfiguration

```

1: Data: _phase_table
2: (start_gid, start_comm_idx, end_gid, end_comm_idx)
3: Per-group state:
4:   idx  $\leftarrow$  0
5:   comm_group  $\leftarrow$  assigned_group
6:   asym_way  $\leftarrow$  way_in_asymmetrical_parallelism
7: Global state:
8:   comm_stage  $\leftarrow$  0
9:   mode  $\in$  {PROVISIONING, DEFAULT}
10: function PHASE_CHANGE_BEFORE
11:   return phase_table[comm_stage].start_gid = comm_group.id and idx = phase_table[comm_stage].start_comm_idx
12: end function
13: function PHASE_CHANGE_AFTER
14:   return phase_table[comm_stage].end_gid = comm_group.id and idx = phase_table[comm_stage].end_comm_idx
15: end function
16: function GET_NEXT_COMM
17:   if PHASE_CHANGE_AFTER then
18:     return (phase_table[comm_stage + 1].start_gid, phase_table[comm_stage + 1].start_comm_idx)
19:   else
20:     return (comm_group.id, idx + 1)
21:   end if
22: end function
23: procedure PRE_COMM(comm_op)
24:   if comm_op is scale_up or management then
25:     select scale-up or CPU front-end network
26:     return
27:   end if
28:   wait till topology is free
29:   shift  $\leftarrow$  PHASE_CHANGE_BEFORE
30:   if mode = DEFAULT then
31:     if shift or comm_op is asymm. then
32:       TOPO_WRITE(comm_group.id, idx, asym_way)
33:     end if
34:   end if
35:   if shift then
36:     comm_stage  $\leftarrow$  comm_stage + 1
37:     set topology busy
38:   end if
39:   idx  $\leftarrow$  idx + 1
40:   select GPU backend network
41:   register POST_COMM callback
42: end procedure
43: procedure POST_COMM(comm_group, comm_op)
44:   shift  $\leftarrow$  PHASE_CHANGE_AFTER
45:   if mode = PROVISIONING then
46:     if shift or comm_op is asymm. then
47:       (n_group_id, n_idx)  $\leftarrow$  GET_NEXT_COMM
48:       TOPO_WRITE(n_group_id, n_idx, asym_way)
49:     end if
50:   end if
51:   if shift then
52:     set topology free
53:   end if
54: end procedure

```

We model store-and-forward instead of cut-through, so a chunk is fully received at each intermediate domain before being re-sent, and an h -hop route pays the per-link latency plus the serialization delay (chunk size divided by link bandwidth) once per hop. A multi-hop

flow may also queue behind other flows at each hop. A multi-hop route thus costs roughly $h \times$ the latency and link bandwidth of a direct one, which is the penalty measured in §5.3.

DP	GPUs	per-rank batch	mean (ms)	p5 (ms)	p50 (ms)	p95 (ms)
16	2,048	192	136.0	9.1	37.9	811.2
32	4,096	96	69.1	4.0	19.3	414.1
64	8,192	48	35.7	1.5	9.9	215.5
128	16,384	24	19.2	0.8	5.2	116.2

Table 5: Window size statistics (mean / p5 / p50 / p95, pooled over all rails) for a GPT-style workload under DP strong scaling (PP=4, TP=32, global batch size 3,072, sequence length 4096, 96 layers) running on B200.

Algorithm 4 Diverged ring assignment per EP group

Require: domains D ($|D| = m$), rails G , candidate orderings C

```

1:  $w(e) \leftarrow 0$  for every pair  $e \in \binom{D}{2}$  ▷ # rails wiring  $e$ 
2: for  $r = 1 \rightarrow G$  do
3:   for all  $\pi \in C$  do
4:      $u(\pi) \leftarrow |\{e \in E(\pi) : w(e) = 0\}|$  ▷ new pairs
5:   end for
6:    $\pi_r \leftarrow \arg \max_{\pi \in C} (u(\pi), -\max_{e \in E(\pi)} w(e))$  ▷ most new; tie: least reuse
7:    $w(e) \leftarrow w(e) + 1 \ \forall e \in E(\pi_r)$ 
8: end for
9: return  $(\pi_1, \dots, \pi_G)$  ▷ ring for each rail

```
